1. Definição inicial do projeto

Sistemas de Informação - Vestibular 2027 - FHO | Fundação Hermíno Ometto
vestibular.fho.edu.br

Sistema de Reserva de Laboratórios e Equipamentos

Educação
Projeto acadêmico
Pequeno porte

Proposta: desenvolver uma solução de software que permita aos professores e funcionários de uma instituição de ensino consultar a disponibilidade de laboratórios e equipamentos e realizar solicitações de reserva.

1.1. Problema identificado

Em algumas instituições de ensino, a reserva de laboratórios e equipamentos pode ser realizada por meio de planilhas, mensagens, e-mails ou solicitações presenciais.

Essa forma de organização pode dificultar o controle das reservas, provocar conflitos de horários, gerar informações desatualizadas e tornar mais demorado o processo de consulta à disponibilidade dos recursos.

Diante dessa situação, surge a necessidade de um sistema que centralize essas informações e facilite o gerenciamento das reservas.

1.2. Solução proposta

O sistema permitirá que professores e funcionários:

- Consultem os laboratórios e equipamentos disponíveis.
- Verifiquem os horários de utilização.
- Solicitem reservas informando data, horário e finalidade.
- Acompanhem a situação das solicitações.
- Consultem o histórico de reservas.

Um usuário responsável pela administração poderá cadastrar laboratórios e equipamentos, analisar solicitações, aprovar ou rejeitar reservas e acompanhar a utilização dos recursos.

1.3. Delimitação do projeto

Para manter o projeto adequado à atividade acadêmica, vamos trabalhar com funcionalidades essenciais.

Dentro do escopo	Fora do escopo
Cadastro de usuários	Aplicativo para celular
Cadastro de laboratórios	Controle de acesso físico
Cadastro de equipamentos	Integração com catracas
Consulta de disponibilidade	Monitoramento em tempo real
Solicitação de reservas	Inteligência artificial
Aprovação e rejeição	Pagamentos
Histórico de reservas	Integração com sistemas externos

Essas limitações ajudam a manter o projeto viável e evitam adicionar funcionalidades que não são necessárias para resolver o problema principal.

1.4. Premissas adotadas

Como o enunciado não apresenta uma instituição específica, utilizaremos uma instituição de ensino fictícia como referência para o desenvolvimento do projeto.

Vamos considerar as seguintes premissas:

1. A instituição possui laboratórios de informática e equipamentos que podem ser utilizados em diferentes atividades acadêmicas.
2. Professores e funcionários são os usuários responsáveis por solicitar reservas.
3. Existe um funcionário responsável por administrar os recursos e analisar as solicitações.
4. Uma reserva precisa ser aprovada antes de ser considerada confirmada.
5. O sistema deve impedir reservas conflitantes para o mesmo laboratório ou equipamento.
6. Os usuários devem conseguir consultar a situação de suas solicitações.

Essas informações são premissas do projeto, não fatos pesquisados sobre uma instituição real.

2. Identificação dos usuários e interessados (Stakeholders)

Antes de definir as funcionalidades, precisamos identificar quem utilizará o sistema e quem será beneficiado por ele.

Stakeholder	Participação no sistema
Professor	Consulta disponibilidade e solicita reservas.
Funcionário	Realiza solicitações e acompanha reservas.
Administrador	Cadastra recursos, gerencia usuários e analisa solicitações.
Instituição de ensino	É beneficiada pela organização e pelo controle dos recursos.

2.1. Como as informações seriam levantadas?

Em um projeto real, poderíamos utilizar algumas técnicas de levantamento de requisitos:

- Entrevistas: conversar com professores e funcionários para entender as dificuldades atuais.
- Questionários: identificar quais funcionalidades seriam mais importantes para os usuários.
- Observação: acompanhar como as reservas são realizadas atualmente.
- Análise documental: verificar planilhas, formulários e documentos utilizados no controle dos recursos.

Como estamos desenvolvendo uma proposta acadêmica, essas técnicas serão apresentadas como métodos que poderiam ser utilizados. Não afirmaremos que entrevistas ou pesquisas foram realizadas.

3. Organização do desenvolvimento

Vamos desenvolver o relatório seguindo os critérios da avaliação, priorizando principalmente UML e padrões de projeto, que juntos representam 19 dos 50 pontos.

Distribuição da pontuação

Contexto e requisitos

5/50

UML e modelagem

12/50

Orientação a objetos

6/50

Padrões de projeto

7/50

Metodologia

4/50

Qualidade, riscos e sustentabilidade

4/50

Comunicação técnica

Minha sugestão é não começar desenhando os diagramas. Primeiro precisamos estabelecer exatamente o que o sistema fará. Assim, os diagramas representarão requisitos já definidos, evitando contradições.

Nossa próxima etapa será a definição dos requisitos funcionais e não funcionais, incluindo identificação, descrição, prioridade e justificativa de cada requisito.

Ao final, teremos uma especificação que servirá de base para todos os diagramas UML e para as decisões de projeto.

2. Requisitos do Sistema

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, vamos definir o que o sistema deverá fazer e quais características deverá apresentar para funcionar adequadamente.

Na Engenharia de Software, os requisitos são utilizados para registrar as necessidades dos usuários e orientar o desenvolvimento do sistema. Eles são divididos principalmente em duas categorias:

- Requisitos funcionais (RF): descrevem as funcionalidades e os serviços que o sistema deverá oferecer.
- Requisitos não funcionais (RNF): descrevem características de qualidade, segurança, desempenho, usabilidade e outras condições que o sistema deverá atender.

Para organizar a documentação, cada requisito receberá um código de identificação, uma descrição, uma prioridade e uma justificativa.

2.1. Critérios de prioridade

Utilizaremos três níveis de prioridade:

Prioridade	Significado
Alta	Funcionalidade ou característica essencial para o funcionamento do sistema.

Média	Importante para melhorar a utilização e o gerenciamento do sistema.
Baixa	Pode ser desenvolvida posteriormente, sem comprometer as funções principais.

A prioridade representa a importância do requisito para o projeto, não necessariamente a ordem em que será implementado.

2.2. Requisitos funcionais (RF)

Os requisitos funcionais representam as ações que os usuários poderão realizar no sistema.

A partir do problema identificado, foram definidos os seguintes requisitos:

ID	Descrição do requisito	Prioridade	Justificativa
RF01	O sistema deverá permitir que os usuários realizem login utilizando e-mail e senha.	Alta	É necessário identificar os usuários e controlar o acesso às funcionalidades.
RF02	O sistema deverá permitir que o administrador cadastre, consulte, altere e desative usuários.	Alta	Permite manter os usuários autorizados e seus dados atualizados.
RF03	O sistema deverá permitir que o administrador cadastre, consulte, altere e desative laboratórios e equipamentos.	Alta	É necessário manter os recursos disponíveis para consulta e reserva.
RF04	O sistema deverá permitir que professores e funcionários consultem os laboratórios e equipamentos cadastrados.	Alta	Facilita a localização dos recursos que podem ser utilizados.

RF05	O sistema deverá permitir a consulta da disponibilidade dos recursos por data e horário.	Alta	Evita que os usuários solicitem reservas sem verificar previamente a disponibilidade.
RF06	O sistema deverá permitir que professores e funcionários solicitem reservas, informando o recurso, a data, o horário e a finalidade da utilização.	Alta	É a principal funcionalidade do sistema e permite organizar as solicitações.
RF07	O sistema deverá verificar se existe outra reserva confirmada ou solicitação pendente para o mesmo recurso e período.	Alta	Evita conflitos entre solicitações e reservas de um mesmo recurso.
RF08	O sistema deverá permitir que o administrador aprove ou rejeite as solicitações de reserva.	Alta	Permite controlar a utilização dos recursos e garantir que as solicitações sejam analisadas.
RF09	O sistema deverá permitir que os usuários consultem o andamento e o histórico de suas solicitações.	Média	Facilita o acompanhamento das reservas e permite consultar informações anteriores.
RF10	O sistema deverá permitir que os usuários cancelem suas solicitações ou reservas, respeitando as regras estabelecidas pela instituição.	Média	Oferece flexibilidade aos usuários e permite liberar recursos que não serão utilizados.

Observação sobre o RF07: a verificação deverá considerar tanto reservas confirmadas quanto solicitações pendentes. Dessa forma, dois usuários não poderão solicitar simultaneamente o mesmo recurso para horários conflitantes enquanto aguardam aprovação.

Também será necessário considerar que uma solicitação rejeitada ou cancelada não deverá bloquear a disponibilidade do recurso.

2.3. Requisitos não funcionais (RNF)

Os requisitos não funcionais definem as características que o sistema deverá apresentar para oferecer uma utilização adequada.

Para este projeto, foram considerados aspectos de segurança, desempenho, usabilidade, compatibilidade, integridade e manutenção.

ID	Descrição do requisito	Prioridade	Justificativa
RNF01	O sistema deverá exigir autenticação por e-mail e senha para permitir o acesso às funcionalidades restritas.	Alta	Protege as informações e impede o acesso de pessoas não autorizadas.
RNF02	O sistema deverá apresentar diferentes permissões de acesso para administradores, professores e funcionários.	Alta	Cada usuário deverá acessar somente as funcionalidades relacionadas ao seu perfil.
RNF03	O sistema deverá apresentar as principais consultas e operações em até 3 segundos, considerando condições normais de utilização.	Média	Proporciona uma experiência mais rápida e evita esperas desnecessárias.
RNF04	O sistema deverá apresentar uma interface simples, organizada e de fácil compreensão, permitindo sua utilização sem conhecimentos técnicos avançados.	Alta	Facilita a utilização por professores e funcionários com diferentes níveis de familiaridade com tecnologia.
RNF05	O sistema deverá funcionar nos principais navegadores atuais,	Média	Permite que os usuários acessem o sistema sem

	como Google Chrome, Microsoft Edge e Mozilla Firefox.		depender de um navegador específico.
RNF06	O sistema deverá manter a integridade das informações, evitando reservas duplicadas e alterações indevidas nos registros.	Alta	Garante a confiabilidade das informações sobre a disponibilidade dos recursos.
RNF07	As senhas dos usuários deverão ser armazenadas de forma protegida, utilizando mecanismos adequados de criptografia por hash.	Alta	Evita o armazenamento de senhas em texto simples e reduz os riscos em caso de acesso indevido aos dados.
RNF08	O sistema deverá permitir sua manutenção e evolução sem exigir alterações desnecessárias em todas as partes da aplicação.	Média	Facilita a correção de problemas e a inclusão de novas funcionalidades no futuro.
RNF09	O sistema deverá proteger os dados pessoais dos usuários e permitir acesso somente às pessoas autorizadas.	Alta	Contribui para a privacidade das informações e para o cumprimento da Lei Geral de Proteção de Dados (LGPD).

Observação: no RNF07, o termo mais adequado é função hash criptográfica para senhas, e não criptografia reversível. O sistema deverá utilizar um mecanismo próprio para armazenamento seguro de senhas, como Argon2 ou bcrypt.

2.4. Regras de negócio

Além dos requisitos, o sistema deverá respeitar algumas regras que determinam como as operações serão realizadas.

ID	Regra de negócio

RN01	Somente usuários cadastrados e autenticados poderão acessar as funcionalidades restritas.
RN02	Somente administradores poderão cadastrar, alterar ou desativar usuários e recursos.
RN03	Uma reserva deverá possuir data, horário de início, horário de término e finalidade.
RN04	Não será permitido reservar o mesmo recurso em períodos que apresentem conflito com outra solicitação pendente ou reserva confirmada.
RN05	Uma solicitação somente será considerada confirmada após a aprovação do administrador.
RN06	O administrador deverá informar o motivo quando rejeitar uma solicitação.
RN07	Usuários não poderão cancelar solicitações ou reservas pertencentes a outras pessoas.
RN08	Recursos desativados não poderão receber novas solicitações de reserva.

Essas regras complementam os requisitos funcionais e ajudam a estabelecer o comportamento esperado do sistema.

2.5. Resumo e priorização dos requisitos

Para facilitar a visualização do que será desenvolvido inicialmente, podemos organizar os requisitos por prioridade.

Requisitos funcionais

7

Alta prioridade

2

Média prioridade

0

Baixa prioridade

Requisitos não funcionais

6

Alta prioridade

3

Média prioridade

0

Baixa prioridade

Total de requisitos

19

10 requisitos funcionais e 9 requisitos não funcionais.

2.6. Considerações sobre os requisitos

A definição dos requisitos permitiu identificar as principais funcionalidades necessárias para solucionar o problema apresentado.

Os requisitos funcionais estabelecem as operações que poderão ser realizadas pelos usuários, enquanto os requisitos não funcionais definem características importantes para garantir segurança, facilidade de utilização e confiabilidade.

As prioridades também ajudam a organizar uma futura implementação, permitindo que as funcionalidades essenciais sejam desenvolvidas primeiro.

É importante destacar que os requisitos apresentados representam uma proposta inicial. Em um projeto real, eles precisariam ser validados com os representantes da instituição de ensino antes do início do desenvolvimento.

2.7. O que já temos e o que vem a seguir

Etapa concluída

- Contexto e problema.
- Solução proposta e delimitação do escopo.
- Identificação dos stakeholders.
- Premissas do projeto.
- Requisitos funcionais.
- Requisitos não funcionais.
- Regras de negócio.

Próxima etapa: Diagrama de Casos de Uso

Vamos identificar os atores, representar graficamente as funcionalidades e descrever dois casos de uso principais:

1. Solicitar reserva de laboratório ou equipamento.
2. Analisar e aprovar ou rejeitar uma solicitação.

Minha recomendação: mantenha esses requisitos como referência para as próximas etapas. Os diagramas UML deverão representar exatamente as funcionalidades e regras que acabamos de definir, evitando adicionar funcionalidades sem justificativa.

3. Diagrama de Casos de Uso

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, vamos utilizar a UML (Unified Modeling Language) para representar visualmente as funcionalidades do sistema e a interação entre os usuários e a aplicação.

O Diagrama de Casos de Uso permite identificar quem utiliza o sistema, quais operações podem ser realizadas e quais são as principais relações entre os usuários e as funcionalidades.

Ele é importante porque apresenta uma visão geral do funcionamento do sistema antes de detalharmos sua estrutura interna.

3.1. Identificação dos atores

Os atores representam pessoas ou entidades externas que interagem com o sistema.

Para este projeto, foram identificados três atores principais:

Ator	Descrição	Principais responsabilidades
Professor	Usuário que utiliza os recursos da instituição para atividades acadêmicas.	Consultar disponibilidade, solicitar reservas e acompanhar solicitações.
Funcionário	Colaborador da instituição que também pode utilizar os recursos.	Consultar disponibilidade, solicitar reservas e acompanhar solicitações.
Administrador	Responsável pelo gerenciamento do sistema e dos recursos disponíveis.	Cadastrar usuários e recursos, consultar solicitações, aprovar ou rejeitar reservas.

Observação: professores e funcionários possuem praticamente as mesmas permissões de utilização. Por isso, serão representados como atores separados no diagrama, mas poderão compartilhar as mesmas funcionalidades.

3.2. Identificação dos casos de uso

Os casos de uso representam as funcionalidades que o sistema disponibilizará aos atores.

A partir dos requisitos definidos anteriormente, foram identificados os seguintes casos:

ID	Caso de uso	Ator responsável

UC01	Realizar login	Professor, Funcionário e Administrador
UC02	Consultar laboratórios e equipamentos	Professor, Funcionário e Administrador
UC03	Consultar disponibilidade	Professor e Funcionário
UC04	Solicitar reserva	Professor e Funcionário
UC05	Consultar solicitações e histórico	Professor e Funcionário
UC06	Cancelar solicitação ou reserva	Professor e Funcionário
UC07	Gerenciar usuários	Administrador
UC08	Gerenciar laboratórios e equipamentos	Administrador
UC09	Consultar solicitações de reserva	Administrador
UC10	Aprovar solicitação	Administrador
UC11	Rejeitar solicitação	Administrador

Esses casos de uso representam as principais funcionalidades previstas para a primeira versão do sistema.

3.3. Diagrama de Casos de Uso

O diagrama abaixo apresenta os três atores e suas interações com as funcionalidades do sistema.

#chatgpt-mermaid-_r_1cp_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(237, 237, 237);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-

dashoffset:0;}}#chatgpt-mermaid-_r_1cp_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1cp_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1cp_ .error-icon{fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_1cp_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_1cp_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_1cp_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_1cp_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_1cp_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_1cp_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1cp_ .marker.cross{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1cp_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_1cp_ p{margin:0;}#chatgpt-mermaid-_r_1cp_ .label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .cluster-label text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .cluster-label span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .cluster-label span p{background-color:transparent;}#chatgpt-mermaid-_r_1cp_ .label text,#chatgpt-mermaid-_r_1cp_ span{fill:rgb(237, 237, 237);color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .node rect,#chatgpt-mermaid-_r_1cp_ .node circle,#chatgpt-mermaid-_r_1cp_ .node ellipse,#chatgpt-mermaid-_r_1cp_ .node polygon,#chatgpt-mermaid-_r_1cp_ .node path{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .rough-node .label text,#chatgpt-mermaid-_r_1cp_ .node .label text,#chatgpt-mermaid-_r_1cp_ .image-shape .label,#chatgpt-mermaid-_r_1cp_ .icon-shape .label{text-anchor:middle;}#chatgpt-mermaid-_r_1cp_ .node .katex path{fill:#000;stroke:#000;stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .rough-node .label,#chatgpt-mermaid-_r_1cp_ .node .label,#chatgpt-mermaid-_r_1cp_ .image-shape .label,#chatgpt-mermaid-_r_1cp_ .icon-shape .label{text-align:center;}#chatgpt-mermaid-_r_1cp_ .node.clickable{cursor:pointer;}#chatgpt-mermaid-_r_1cp_ .root .anchor path{fill:rgb(175, 175, 175)!important;stroke-width:0;stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1cp_ .arrowheadPath{fill:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1cp_ .edgePath .path{stroke:rgb(175, 175, 175);stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .flowchart-link{stroke:rgb(175, 175, 175);fill:none;}#chatgpt-mermaid-_r_1cp_ .edgeLabel{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_1cp_ .edgeLabel p{background-color:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1cp_ .edgeLabel rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-

mermaid-_r_1cp_ .labelBkg{background-color:rgba(0, 0, 0, 0.5);}#chatgpt-mermaid-_r_1cp_ .cluster rect{fill:rgb(48, 48, 48);stroke:rgba(255, 255, 255, 0.15);stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .cluster text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ .cluster span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ div.mermaidTooltip{position:absolute;text-align:center;max-width:200px;padding:2px;font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:12px;background:rgb(48, 48, 48);border:1px solid rgba(255, 255, 255, 0.15);border-radius:2px;pointer-events:none;z-index:100;}#chatgpt-mermaid-_r_1cp_ .flowchartTitleText{text-anchor:middle;font-size:18px;fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1cp_ rect.text{fill:none;stroke-width:0;}#chatgpt-mermaid-_r_1cp_ .icon-shape,#chatgpt-mermaid-_r_1cp_ .image-shape{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_1cp_ .icon-shape p,#chatgpt-mermaid-_r_1cp_ .image-shape p{background-color:rgb(0, 0, 0);padding:2px;}#chatgpt-mermaid-_r_1cp_ .icon-shape .label rect,#chatgpt-mermaid-_r_1cp_ .image-shape .label rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1cp_ .label-icon{display:inline-block;height:1em;overflow:visible;vertical-align:-0.125em;}#chatgpt-mermaid-_r_1cp_ .node .label-icon path{fill:currentColor;stroke:revert;stroke-width:revert;}#chatgpt-mermaid-_r_1cp_ .node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_1cp_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_1cp_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_1cp_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_1cp_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_1cp_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1cp_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_1cp_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1cp_ .node text{font-size:14px;font-weight:600;letter-spacing:normal;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_1cp_ .edgeLabels text{font-size:13px;font-weight:600;letter-spacing:-0.08px;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_1cp_ .node tspan[font-weight="normal"],#chatgpt-mermaid-_r_1cp_ .edgeLabels tspan[font-weight="normal"]{font-weight:600;}#chatgpt-mermaid-_r_1cp_ .edgeLabel .label rect{opacity:1;rx:13px;ry:13px;fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-

width:1px;}#chatgpt-mermaid-_r_1cp_ .node rect,#chatgpt-mermaid-_r_1cp_ .node circle,#chatgpt-mermaid-_r_1cp_ .node ellipse,#chatgpt-mermaid-_r_1cp_ .node polygon,#chatgpt-mermaid-_r_1cp_ .node path{fill:rgb(0, 40, 77);stroke:rgba(255, 255, 255, 0.1);stroke-width:1px;}#chatgpt-mermaid-_r_1cp_ .node rect{rx:16px;ry:16px;}#chatgpt-mermaid-_r_1cp_ .mermaid-decision .label-container{fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-dasharray:2,2;}#chatgpt-mermaid-_r_1cp_ .edgePaths .flowchart-link{stroke:rgb(175, 175, 175);stroke-width:1px;stroke-linecap:round;stroke-linejoin:round;}#chatgpt-mermaid-_r_1cp_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1cp_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}Sistema de Reserva de Laboratórios e EquipamentosUC01 - Realizar loginUC02 - Consultar laboratóriose equipamentosUC03 - ConsultardisponibilidadeUC04 - Solicitar reservaUC05 - Consultar solicitações ehistóricoUC06 - Cancelar solicitação oureservaUC07 - Gerenciar usuáriosUC08 - Gerenciar laboratórios eequipamentosUC09 - Consultar solicitaçõesde reservaUC10 - Aprovar solicitaçãoUC11 - Rejeitar solicitação Professor Funcionário Administrador
Figura 1 – Diagrama de Casos de Uso do Sistema de Reserva de Laboratórios e Equipamentos. Fonte: Elaborado pelo autor (2026).

Como interpretar o diagrama:

- Os bonecos representam os atores que interagem com o sistema.
- Os elementos arredondados representam os casos de uso.
- As linhas representam as associações entre os atores e as funcionalidades.
- O retângulo maior representa o limite do sistema, ou seja, as funcionalidades que fazem parte da aplicação.

O login aparece associado aos três atores, pois todos precisam se autenticar para acessar as funcionalidades restritas.

3.4. Descrição dos principais casos de uso

Além do diagrama, é importante descrever detalhadamente como as principais funcionalidades deverão funcionar.

Foram escolhidos dois casos de uso centrais: solicitar uma reserva e analisar uma solicitação.

UC04 – Solicitar reserva

Elemento	Descrição
Identificação	UC04
Nome	Solicitar reserva
Ator principal	Professor ou Funcionário
Objetivo	Solicitar a utilização de um laboratório ou equipamento.
Pré-condições	O usuário deve estar cadastrado, autenticado e ativo no sistema.
Pós-condições	A solicitação fica registrada com o status Pendente, caso seja válida.

Fluxo principal:

1. O usuário acessa o sistema utilizando seu login.
2. O usuário consulta os laboratórios e equipamentos disponíveis.
3. O usuário seleciona o recurso que deseja reservar.
4. O sistema apresenta as opções de data e horário.
5. O usuário informa a data, o horário de início, o horário de término e a finalidade da utilização.
6. O sistema verifica se existe conflito com outra solicitação pendente ou reserva confirmada.
7. Caso não exista conflito, o sistema registra a solicitação.
8. O sistema apresenta uma mensagem informando que a solicitação foi registrada e está aguardando aprovação.

Fluxos alternativos:

- A1 – Recurso indisponível: se existir conflito de horário, o sistema informa que o recurso não está disponível e solicita que o usuário escolha outro período.
- A2 – Informações incompletas: se algum campo obrigatório não for preenchido, o sistema solicita que o usuário complete as informações.

- A3 – Recurso desativado: se o recurso estiver desativado, o sistema impede a solicitação e informa a situação ao usuário.

Resultado esperado: a solicitação é registrada e fica disponível para análise do administrador.

UC10 – Aprovar ou rejeitar solicitação

Elemento	Descrição
Identificação	UC10 e UC11
Nome	Analisar solicitação de reserva
Ator principal	Administrador
Objetivo	Analisar uma solicitação e decidir se ela será aprovada ou rejeitada.
Pré-condições	O administrador deve estar autenticado e existir uma solicitação pendente.
Pós-condições	A solicitação recebe o status Aprovada ou Rejeitada.

Fluxo principal:

1. O administrador acessa o sistema.
2. O administrador consulta a lista de solicitações pendentes.
3. O sistema apresenta as informações da solicitação, incluindo usuário, recurso, data, horário e finalidade.
4. O administrador analisa as informações.
5. O administrador seleciona a opção de aprovar ou rejeitar.
6. Caso aprove, o sistema registra a aprovação e altera o status da solicitação para Aprovada.
7. Caso rejeite, o sistema solicita o preenchimento do motivo da rejeição e altera o status para Rejeitada.
8. O sistema atualiza as informações da solicitação.

Fluxos alternativos:

- A1 – Solicitação já analisada: caso a solicitação não esteja mais pendente, o sistema informa que ela já foi processada e impede uma nova análise.
- A2 – Motivo não informado: caso o administrador escolha rejeitar sem informar o motivo, o sistema solicita o preenchimento desse campo.
- A3 – Conflito identificado: caso exista uma reserva confirmada conflitante antes da aprovação, o sistema impede a confirmação e informa o problema ao administrador.

Resultado esperado: a solicitação recebe uma decisão e seu status fica atualizado para consulta pelo usuário.

3.5. Relação entre os casos de uso e os requisitos

É importante verificar se as funcionalidades representadas no diagrama correspondem aos requisitos definidos na etapa anterior.

Caso de uso	Requisitos relacionados
UC01 – Realizar login	RF01, RNF01 e RNF02
UC02 – Consultar laboratórios e equipamentos	RF04
UC03 – Consultar disponibilidade	RF05
UC04 – Solicitar reserva	RF06, RF07 e RN03
UC05 – Consultar solicitações e histórico	RF09
UC06 – Cancelar solicitação ou reserva	RF10
UC07 – Gerenciar usuários	RF02

UC08 – Gerenciar laboratórios e equipamentos	RF03
UC09 – Consultar solicitações de reserva	RF08
UC10 – Aprovar solicitação	RF08
UC11 – Rejeitar solicitação	RF08 e RN06

Essa relação permite verificar se os requisitos funcionais estão representados pelas funcionalidades do sistema.

3.6. Considerações sobre o Diagrama de Casos de Uso

A elaboração do Diagrama de Casos de Uso permitiu identificar os principais usuários do sistema e as funcionalidades necessárias para atender às suas necessidades.

Foi possível perceber que professores e funcionários compartilham funcionalidades relacionadas à consulta e solicitação de reservas, enquanto o administrador possui responsabilidades específicas de gerenciamento e aprovação.

As descrições dos dois principais casos de uso também ajudaram a compreender melhor o comportamento esperado do sistema, incluindo as condições necessárias para executar cada operação e as situações que podem impedir sua conclusão.

Dessa forma, o diagrama servirá como referência para a próxima etapa de modelagem, na qual serão identificadas as classes, seus atributos, métodos e relacionamentos.

3.7. Checklist da etapa

Diagrama de Casos de Uso

Concluído

- Identificação dos atores.
- Identificação dos casos de uso.
- Representação gráfica do sistema.
- Descrição do caso de uso Solicitar reserva.

- Descrição do caso de uso Analisar solicitação.
- Fluxos principais e alternativos.
- Relação com os requisitos.

Próxima etapa: Diagrama de Classes.

Nela, vamos identificar as principais classes do sistema, seus atributos, métodos e relacionamentos. Algumas classes que provavelmente farão parte do modelo são `Usuario`, `Professor`, `Funcionario`, `Administrador`, `Laboratorio`, `Equipamento` e `Reserva`.

Essa etapa será especialmente importante porque também servirá de base para explicar os conceitos de orientação a objetos e os padrões de projeto que utilizaremos posteriormente.

4. Diagrama de Classes

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, vamos desenvolver o Diagrama de Classes, que representa a estrutura do sistema e demonstra como suas principais partes estarão organizadas.

Diferentemente do Diagrama de Casos de Uso, que apresenta as funcionalidades e os usuários, o Diagrama de Classes mostra os elementos que compõem o sistema, incluindo:

- Classes e seus respectivos nomes.
- Atributos, que representam as informações armazenadas pelos objetos.
- Métodos, que representam as ações que os objetos podem executar.
- Relacionamentos entre as classes.
- Herança, associação e multiplicidade.

Esse diagrama será importante para demonstrar como os conceitos de orientação a objetos poderão ser utilizados na construção do sistema.

4.1. Identificação das classes

Com base nos requisitos e nos casos de uso definidos anteriormente, foram identificadas as principais classes necessárias para o funcionamento do sistema.

Classe	Responsabilidade

Usuario	Representar os usuários cadastrados no sistema.
Professor	Representar os usuários que possuem o perfil de professor.
Funcionario	Representar os funcionários da instituição.
Administrador	Representar os usuários responsáveis pelo gerenciamento do sistema.
Recurso	Representar os recursos que podem ser reservados.
Laboratorio	Representar os laboratórios disponíveis na instituição.
Equipamento	Representar os equipamentos disponíveis para utilização.
Reserva	Armazenar as informações das solicitações e reservas.
ServicoReserva	Controlar as operações relacionadas às solicitações, aprovações, cancelamentos e verificações de disponibilidade.

A classe `Usuario` será uma classe abstrata, pois servirá como base para os diferentes tipos de usuários. Dessa forma, não será necessário criar diretamente um objeto genérico dessa classe.

Da mesma maneira, `Recurso` será abstrata, pois os recursos concretos do sistema serão representados por laboratórios e equipamentos.

4.2. Atributos e métodos das classes

Antes de construir o diagrama, precisamos definir quais informações e operações serão responsabilidade de cada classe.

Classe Usuario

Será responsável por armazenar as informações comuns aos usuários do sistema.

Atributos:

- `id`: identificação do usuário.
- `nome`: nome completo.
- `email`: endereço de e-mail.
- `senhaHash`: senha armazenada de forma protegida.
- `ativo`: indica se o usuário está ativo.

Métodos:

- `realizarLogin()`: realizar a autenticação do usuário.
- `atualizarDados()`: atualizar as informações cadastrais.

Classe Professor

Representará os professores da instituição.

Atributos:

- `matricula`: número de identificação do professor.

Métodos:

- `solicitarReserva()`: solicitar a utilização de um recurso.
- `consultarHistorico()`: consultar suas solicitações anteriores.

Classe Funcionario

Representará os funcionários que poderão solicitar a utilização dos recursos.

Atributos:

- `matricula`: identificação do funcionário.
- `setor`: setor em que trabalha.

Métodos:

- `solicitarReserva()`: solicitar uma reserva.
- `consultarHistorico()`: consultar suas solicitações anteriores.

Classe Administrador

Representará os responsáveis pelo gerenciamento do sistema.

Atributos:

- `nivelAcesso`: nível de permissão administrativa.

Métodos:

- `gerenciarUsuarios()`: administrar os usuários cadastrados.
- `gerenciarRecursos()`: administrar laboratórios e equipamentos.
- `analisarSolicitacao()`: analisar uma solicitação de reserva.

Classe Recurso

Será uma classe abstrata que reunirá as informações comuns aos laboratórios e equipamentos.

Atributos:

- `id`: identificação do recurso.
- `nome`: nome do recurso.
- `descricao`: descrição do recurso.
- `ativo`: indica se o recurso está disponível para novas solicitações.

Métodos:

- `consultarDisponibilidade()`: verificar a disponibilidade do recurso.

Classe Laboratorio

Representará os laboratórios cadastrados na instituição.

Atributos:

- `capacidade`: quantidade máxima de pessoas.
- `localizacao`: localização do laboratório.

Métodos:

- `consultarCapacidade()`: consultar a capacidade do laboratório.

Classe Equipamento

Representará os equipamentos que poderão ser reservados.

Atributos:

- `patrimonio`: número de identificação patrimonial.
- `estadoConservacao`: condição de conservação do equipamento.

Métodos:

- `consultarEstado()`: consultar o estado de conservação.

Classe Reserva

Será responsável por armazenar as informações relacionadas às solicitações e reservas.

Atributos:

- `id`: identificação da reserva.
- `data`: data de utilização.
- `horaInicio`: horário inicial.
- `horaFim`: horário final.
- `finalidade`: motivo da utilização.
- `status`: situação atual da solicitação.
- `motivoRejeicao`: justificativa caso a solicitação seja rejeitada.

Métodos:

- `criarReserva()`: registrar uma nova solicitação.
- `aprovar()`: aprovar a solicitação.
- `rejeitar()`: rejeitar a solicitação.
- `cancelar()`: cancelar a solicitação ou reserva.

Classe ServicoReserva

Será responsável por organizar as operações relacionadas às reservas, evitando que todas as regras fiquem concentradas em uma única classe.

Métodos:

- `solicitarReserva()`: receber e processar uma solicitação.
- `verificarConflito()`: verificar se existe outra solicitação pendente ou reserva aprovada no mesmo período.
- `aprovarReserva()`: processar a aprovação de uma solicitação.
- `rejeitarReserva()`: processar a rejeição de uma solicitação.

Essa separação permite que a classe `Reserva` cuide das informações e do estado da reserva, enquanto `ServicoReserva` coordena as operações e verifica as regras necessárias.

4.3. Diagrama de Classes

Agora que identificamos as classes, seus atributos e métodos, podemos representar graficamente a estrutura do sistema.

O diagrama abaixo apresenta as classes, os relacionamentos de herança, as associações e as multiplicidades.

#chatgpt-mermaid-_r_1d9_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(237, 237, 237);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_1d9_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1d9_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1d9_ .error-icon{fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_1d9_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_1d9_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_1d9_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_1d9_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_1d9_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_1d9_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_1d9_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1d9_ .marker.cross{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1d9_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_1d9_ p{margin:0;}#chatgpt-mermaid-_r_1d9_ g.classGroup text{fill:rgb(31, 78, 148);stroke:none;font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:10px;}#chatgpt-mermaid-_r_1d9_ g.classGroup text .title{font-weight:bolder;}#chatgpt-mermaid-_r_1d9_ .cluster-label text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .cluster-label span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .cluster-label span p{background-color:transparent;}#chatgpt-mermaid-_r_1d9_ .cluster rect{fill:rgb(48, 48, 48);stroke:rgba(255, 255, 255, 0.15);stroke-width:1px;}#chatgpt-mermaid-_r_1d9_ .cluster text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .cluster span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .nodeLabel,#chatgpt-mermaid-_r_1d9_ .edgeLabel{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .noteLabel .nodeLabel,#chatgpt-mermaid-_r_1d9_ .noteLabel

.edgeLabel{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .edgeLabel .label rect{fill:rgb(9, 23, 44);}#chatgpt-mermaid-_r_1d9_ .label text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1d9_ .labelBkg{background:rgb(9, 23, 44);}#chatgpt-mermaid-_r_1d9_ .edgeLabel .label span{background:rgb(9, 23, 44);}#chatgpt-mermaid-_r_1d9_ .classTitle{font-weight:bolder;}#chatgpt-mermaid-_r_1d9_ .node rect,#chatgpt-mermaid-_r_1d9_ .node circle,#chatgpt-mermaid-_r_1d9_ .node ellipse,#chatgpt-mermaid-_r_1d9_ .node polygon,#chatgpt-mermaid-_r_1d9_ .node path{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);stroke-width:1;}#chatgpt-mermaid-_r_1d9_ .divider{stroke:rgb(31, 78, 148);stroke-width:1;}#chatgpt-mermaid-_r_1d9_ g.clickable{cursor:pointer;}#chatgpt-mermaid-_r_1d9_ g.classGroup rect{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_1d9_ g.classGroup line{stroke:rgb(31, 78, 148);stroke-width:1;}#chatgpt-mermaid-_r_1d9_ .classLabel .box{stroke:none;stroke-width:0;fill:rgb(9, 23, 44);opacity:0.5;}#chatgpt-mermaid-_r_1d9_ .classLabel .label{fill:rgb(31, 78, 148);font-size:10px;}#chatgpt-mermaid-_r_1d9_ .relation{stroke:rgb(175, 175, 175);stroke-width:1;fill:none;}#chatgpt-mermaid-_r_1d9_ .dashed-line{stroke-dasharray:3;}#chatgpt-mermaid-_r_1d9_ .dotted-line{stroke-dasharray:1 2;}#chatgpt-mermaid-_r_1d9_ [id$="-compositionStart"],#chatgpt-mermaid-_r_1d9_ .composition{fill:rgb(175, 175, 175)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-compositionEnd"],#chatgpt-mermaid-_r_1d9_ .composition{fill:rgb(175, 175, 175)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-dependencyStart"],#chatgpt-mermaid-_r_1d9_ .dependency{fill:rgb(175, 175, 175)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-dependencyEnd"],#chatgpt-mermaid-_r_1d9_ .dependency{fill:rgb(175, 175, 175)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-extensionStart"],#chatgpt-mermaid-_r_1d9_ .extension{fill:transparent!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-extensionEnd"],#chatgpt-mermaid-_r_1d9_ .extension{fill:transparent!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-aggregationStart"],#chatgpt-mermaid-_r_1d9_ .aggregation{fill:transparent!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-aggregationEnd"],#chatgpt-mermaid-_r_1d9_ .aggregation{fill:transparent!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-lollipopStart"],#chatgpt-mermaid-_r_1d9_ .lollipop{fill:rgb(9, 23, 44)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ [id$="-lollipopEnd"],#chatgpt-mermaid-_r_1d9_ .lollipop{fill:rgb(9, 23, 44)!important;stroke:rgb(175, 175, 175)!important;stroke-width:1;}#chatgpt-mermaid-_r_1d9_ .edgeTerminals{font-size:11px;line-height:initial;}#chatgpt-mermaid-_r_1d9_ .classTitleText{text-anchor:middle;font-size:18px;fill:rgb(237, 237, 237);}#chatgpt-

mermaid-_r_1d9_ .edgeLabel[data-look="neo"]{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_1d9_ .edgeLabel[data-look="neo"] p{background-color:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1d9_ .edgeLabel[data-look="neo"] rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1d9_ .label-icon{display:inline-block;height:1em;overflow:visible;vertical-align:-0.125em;}#chatgpt-mermaid-_r_1d9_ .node .label-icon path{fill:currentColor;stroke:revert;stroke-width:revert;}#chatgpt-mermaid-_r_1d9_ .node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_1d9_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_1d9_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_1d9_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_1d9_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_1d9_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1d9_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_1d9_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1d9_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}«abstract»Usuario-int id-String nome-String email-String senhaHash-boolean ativo+realizarLogin()+atualizarDados()Professor-String matricula+solicitarReserva()+consultarHistorico()Funcionario-String matricula-String setor+solicitarReserva()+consultarHistorico()Administrador-String nivelAcesso+gerenciarUsuarios()+gerenciarRecursos()+analisarSolicitacao()«abstract»Recurso-int id-String nome-String descricao-boolean ativo+consultarDisponibilidade()Laboratorio-int capacidade-String localizacao+consultarCapacidade()Equipamento-String patrimonio-String estadoConservacao+consultarEstado()Reserva-int id-Date data-Time horaInicio-Time horaFim-String finalidade-StatusReserva status-String motivoRejeicao+criarReserva()+aprovar()+rejeitar()+cancelar()«enumeration»StatusReservaPENDENTEAPROVADAREJEITADACANCELADAServicoReserva+solicitarReserva()+verificarConflito()+aprovarReserva()+rejeitarReserva()solicitarecebeanalisapossuigerenciaverifica110..10..*0..*0..*

Figura 2 – Diagrama de Classes do Sistema de Reserva de Laboratórios e Equipamentos. Fonte: Elaborado pelo autor (2026).

4.4. Explicação dos relacionamentos

O diagrama apresenta diferentes tipos de relacionamentos entre as classes.

1. Herança

A herança permite que uma classe aproveite atributos e métodos de outra classe.

No projeto, temos dois exemplos:

- `Professor`, `Funcionario` e `Administrador` herdam as características da classe `Usuario`.
- `Laboratorio` e `Equipamento` herdam as características da classe `Recurso`.

Isso evita repetir informações e facilita a organização do sistema.

2. Associação

A associação representa uma relação entre duas classes.

No projeto:

- Um usuário pode realizar várias reservas.
- Cada reserva pertence a um único usuário solicitante.
- Um recurso pode estar relacionado a várias reservas ao longo do tempo.
- Cada reserva está associada a um único recurso.
- Um administrador pode analisar várias reservas.
- Uma reserva pode ter no máximo um administrador responsável pela análise.

3. Dependência

A classe `ServicoReserva` utiliza informações e operações das classes `Reserva` e `Recurso` para executar suas responsabilidades.

Por exemplo, quando um usuário solicita uma reserva, o serviço verifica se o recurso está ativo e se existe conflito de horário antes de registrar a solicitação.

4. Enumeração

A enumeração `StatusReserva` representa os possíveis estados de uma solicitação:

- `PENDENTE`: aguardando análise.
- `APROVADA`: solicitação aceita.
- `REJEITADA`: solicitação recusada.
- `CANCELADA`: solicitação ou reserva cancelada.

O uso de uma enumeração evita que sejam utilizados diferentes textos para representar a mesma situação.

4.5. Explicação das multiplicidades

As multiplicidades indicam quantos objetos de uma classe podem estar relacionados a objetos de outra classe.

Multiplicidade	Significado
1	Exatamente um objeto.
0..1	Nenhum ou um objeto.
0..*	Nenhum ou vários objetos.
1..*	Um ou vários objetos.

No nosso sistema, temos os seguintes exemplos:

- Usuario – Reserva: um usuário pode possuir várias reservas, mas cada reserva deve possuir um único solicitante.
- Recurso – Reserva: um laboratório ou equipamento pode receber várias reservas em datas e horários diferentes, mas cada reserva corresponde a um único recurso.
- Administrador – Reserva: uma solicitação pode ainda não ter um administrador responsável pela análise ou pode ter um administrador responsável. Um administrador pode analisar várias solicitações.

4.6. Aplicação dos conceitos de orientação a objetos

A estrutura proposta também permite identificar como os principais conceitos de orientação a objetos serão utilizados.

Conceito	Aplicação no projeto

Abstração	As classes `Usuario` e `Recurso` representam características comuns dos seus respectivos tipos.
Encapsulamento	Os atributos das classes são privados e devem ser acessados por métodos apropriados.
Herança	Professor, Funcionário e Administrador herdam características de Usuario. Laboratorio e Equipamento herdam de Recurso.
Polimorfismo	As classes Laboratorio e Equipamento podem apresentar comportamentos específicos para operações definidas na classe Recurso.
Coesão	Cada classe possui responsabilidades relacionadas à sua finalidade.
Baixo acoplamento	O ServicoReserva concentra as operações de gerenciamento, evitando que as regras fiquem espalhadas pelas demais classes.

Um exemplo de aplicação é a classe `Recurso`: tanto um laboratório quanto um equipamento possuem nome, descrição e situação de atividade. Entretanto, cada um apresenta informações específicas, como capacidade e localização para laboratórios e patrimônio e estado de conservação para equipamentos.

4.7. Considerações sobre o Diagrama de Classes

A elaboração do Diagrama de Classes permitiu organizar a estrutura do Sistema de Reserva de Laboratórios e Equipamentos, identificando as principais entidades, suas características e responsabilidades.

A utilização de herança permite compartilhar informações entre classes semelhantes, enquanto as associações representam as relações entre usuários, recursos e reservas.

Também foi proposta a classe `ServicoReserva`, responsável por organizar as operações relacionadas às solicitações e verificar possíveis conflitos de horários.

Essa organização busca facilitar o entendimento, a manutenção e uma futura implementação do sistema, evitando concentrar todas as funcionalidades em uma única classe.

É importante destacar que o diagrama representa uma proposta inicial de modelagem. Alguns detalhes poderão ser ajustados durante o desenvolvimento, conforme as necessidades identificadas.

4.8. Checklist da etapa

Diagrama de Classes

Concluído

- Identificação das principais classes.
- Definição dos atributos.
- Definição dos métodos.
- Representação da herança.
- Representação das associações.
- Definição das multiplicidades.
- Identificação das responsabilidades.
- Relação com os conceitos de orientação a objetos.

4.9. Próxima etapa: Diagrama de Sequência

Agora vamos representar como os objetos do sistema interagem entre si para executar uma funcionalidade.

Minha sugestão é utilizar como exemplo a operação Solicitar Reserva, mostrando a sequência de interações entre:

1. Professor ou funcionário.
2. Sistema.
3. Serviço de reservas.
4. Recurso.
5. Reserva.

Também vamos desenvolver o Diagrama de Atividades para representar o fluxo de funcionamento de uma solicitação, desde a escolha do recurso até a aprovação ou rejeição pelo administrador.

Uma observação importante para a continuidade: o diagrama atual considera que cada solicitação reserva um único recurso (um laboratório ou um equipamento). Vamos manter essa regra nas próximas etapas para preservar a simplicidade e a coerência do projeto.

5. Diagrama de Sequência

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, vamos desenvolver o Diagrama de Sequência, utilizando a UML (Unified Modeling Language) para representar como os diferentes objetos do sistema interagem durante a execução de uma funcionalidade.

Enquanto o Diagrama de Casos de Uso apresenta as funcionalidades disponíveis e o Diagrama de Classes mostra a estrutura do sistema, o Diagrama de Sequência demonstra a ordem em que as operações acontecem.

Ele permite visualizar:

- Quem inicia uma operação.
- Quais objetos participam do processo.
- Quais mensagens são trocadas entre os objetos.
- A ordem em que as ações acontecem.
- Quais decisões podem alterar o fluxo de execução.

Para este projeto, vamos utilizar como exemplo a funcionalidade Solicitar Reserva, pois ela representa uma das principais operações do sistema.

5.1. Identificação dos participantes

Para representar o processo, foram identificados os seguintes participantes:

Participante	Responsabilidade
Professor/Funcionário	Iniciar a solicitação e informar os dados da reserva.
Sistema	Receber as informações e apresentar os resultados ao usuário.
ServicoReserva	Processar a solicitação e verificar as regras de negócio.
Recurso	Fornecer informações sobre sua disponibilidade e situação.

Reserva	Representar e armazenar a solicitação realizada.

Esses participantes foram definidos com base nas classes apresentadas no Diagrama de Classes.

5.2. Descrição do funcionamento

O processo começa quando um professor ou funcionário acessa o sistema e escolhe a opção de solicitar uma reserva.

Em seguida, o usuário seleciona um laboratório ou equipamento, informa a data, o horário de início, o horário de término e a finalidade da utilização.

O sistema encaminha essas informações para o serviço responsável pelo gerenciamento das reservas.

O serviço verifica se o recurso está ativo e se existe alguma solicitação pendente ou reserva aprovada que apresente conflito de horário.

Caso não exista conflito, uma nova solicitação é registrada com o status Pendente. Caso contrário, o sistema informa que o recurso não está disponível para aquele período.

5.3. Diagrama de Sequência – Solicitar Reserva

O diagrama abaixo representa a sequência de interações entre os participantes durante uma solicitação de reserva.

ReservaRecursoServicoReservaSistemaReservaRecursoServicoReservaSistema#chatgpt-mermaid-_r_16u_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(237, 237, 237);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_16u_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_16u_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_16u_ .error-icon{fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_16u_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237, 237);}#chatgpt-mermaid-_r_16u_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_16u_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_16u_ .edge-pattern-solid{stroke-

dasharray:0;}#chatgpt-mermaid-_r_16u_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_16u_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_16u_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_16u_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ .marker.cross{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_16u_ p{margin:0;}#chatgpt-mermaid-_r_16u_ .actor{stroke:rgb(31, 78, 148);fill:rgb(9, 23, 44);stroke-width:1;}#chatgpt-mermaid-_r_16u_ rect.actor.outer-path[data-look="neo"]{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ rect.note[data-look="neo"]{stroke:rgb(58, 132, 63);fill:rgb(48, 48, 48);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ text.actor>tspan{fill:rgb(237, 237, 237);stroke:none;}#chatgpt-mermaid-_r_16u_ .actor-line{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ .innerArc{stroke-width:1.5;stroke-dasharray:none;}#chatgpt-mermaid-_r_16u_ .messageLine0{stroke-width:1.5;stroke-dasharray:none;stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ .messageLine1{stroke-width:1.5;stroke-dasharray:2,2;stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ [id$="-arrowhead"] path{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ .sequenceNumber{fill:#505050;}#chatgpt-mermaid-_r_16u_ [id$="-sequencenumber"]{fill:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ [id$="-crosshead"] path{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_16u_ .messageText{fill:rgb(237, 237, 237);stroke:none;}#chatgpt-mermaid-_r_16u_ .labelBox{stroke:rgba(255, 255, 255, 0.15);fill:rgb(0, 0, 0);filter:none;}#chatgpt-mermaid-_r_16u_ .labelText,#chatgpt-mermaid-_r_16u_ .labelText>tspan{fill:rgb(237, 237, 237);stroke:none;}#chatgpt-mermaid-_r_16u_ .loopText,#chatgpt-mermaid-_r_16u_ .loopText>tspan{fill:rgb(237, 237, 237);stroke:none;}#chatgpt-mermaid-_r_16u_ .sectionTitle,#chatgpt-mermaid-_r_16u_ .sectionTitle>tspan{fill:rgb(237, 237, 237);stroke:none;}#chatgpt-mermaid-_r_16u_ .loopLine{stroke-width:2px;stroke-dasharray:2,2;stroke:rgba(255, 255, 255, 0.15);fill:rgba(255, 255, 255, 0.15);}#chatgpt-mermaid-_r_16u_ .note{stroke:rgb(58, 132, 63);fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_16u_ .noteText,#chatgpt-mermaid-_r_16u_ .noteText>tspan{fill:rgb(237, 237, 237);stroke:none;font-weight:normal;}#chatgpt-mermaid-_r_16u_ .activation0{fill:rgb(48, 48, 48);stroke:hsl(0, 0%, 8.8235294118%);}#chatgpt-mermaid-_r_16u_ .activation1{fill:rgb(48, 48, 48);stroke:hsl(0, 0%, 8.8235294118%);}#chatgpt-mermaid-_r_16u_ .activation2{fill:rgb(48, 48, 48);stroke:hsl(0, 0%, 8.8235294118%);}#chatgpt-mermaid-_r_16u_ .actorPopupMenu{position:absolute;}#chatgpt-mermaid-_r_16u_ .actorPopupMenuPanel{position:absolute;fill:rgb(9, 23, 44);box-shadow:0px 8px 16px 0px rgba(0,0,0,0.2);filter:drop-shadow(3px 5px 2px rgb(0 0 0 / 0.4));}#chatgpt-

mermaid-_r_16u_ .actor-man circle,#chatgpt-mermaid-_r_16u_ line{fill:rgb(9, 23, 44);stroke-width:2px;}#chatgpt-mermaid-_r_16u_ g rect.rect{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_16u_ .node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_16u_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_16u_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_16u_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_16u_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_16u_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_16u_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_16u_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_16u_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_16u_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}alt[Existe conflito de horário][Não existe conflito]alt[Recurso desativado][Recurso ativo]Professor/FuncionárioAcessa a opção Solicitar ReservaApresenta os recursos disponíveisSeleciona recurso e informa os dadossolicitarReserva(dados)consultarDisponibilidade()Retorna situação do recursoInformar que o recurso está indisponívelExibir mensagem de indisponibilidadeverificarConflito()Informar conflito de horárioSolicitar escolha de outro horáriocriarReserva()Reserva registrada com status PENDENTEConfirmar registro da solicitaçãoExibir mensagem de sucessoProfessor/Funcionário
Figura 3 – Diagrama de Sequência da funcionalidade Solicitar Reserva. Fonte: Elaborado pelo autor (2026).

5.4. Explicação da sequência de execução

O diagrama apresenta o processo dividido em algumas etapas:

1. Início da solicitação

O professor ou funcionário acessa a funcionalidade de reserva e informa os dados necessários para solicitar a utilização de um laboratório ou equipamento.

2. Envio das informações

O sistema recebe os dados e encaminha a solicitação para a classe `ServicoReserva`, responsável por organizar o processo.

3. Verificação do recurso

O serviço consulta a situação do recurso para verificar se ele está ativo e pode receber novas solicitações.

4. Verificação de conflitos

Caso o recurso esteja ativo, o serviço verifica se existe outra solicitação pendente ou reserva aprovada para o mesmo período.

5. Registro da solicitação

Se não existir conflito, o sistema cria uma nova reserva e registra sua situação inicial como `PENDENTE`.

6. Retorno ao usuário

Por último, o sistema apresenta uma mensagem informando que a solicitação foi registrada e está aguardando aprovação.

Se o recurso estiver desativado ou apresentar conflito de horário, o sistema informa o problema e não registra a solicitação.

5.5. Elementos UML utilizados

No diagrama, podemos identificar alguns elementos importantes da UML:

Elemento	Significado no projeto
Ator	Representa o professor ou funcionário que inicia a operação.
Participantes	Representam os objetos envolvidos na execução.
Linhas de vida	Linhas verticais que indicam a existência dos participantes durante o processo.
Mensagens	Setas que representam as chamadas e respostas entre os participantes.

Fragmento `alt`	Representa condições alternativas, como recurso disponível ou indisponível.
Retorno	Representa a resposta enviada após a execução de uma operação.

5.6. Relação com o Diagrama de Classes

O Diagrama de Sequência foi elaborado considerando as classes definidas anteriormente.

As principais relações são:

- `Professor` e `Funcionario`: representam os usuários que iniciam a solicitação.
- `ServicoReserva`: recebe os dados e executa as verificações necessárias.
- `Recurso`: fornece informações sobre sua situação.
- `Reserva`: representa a solicitação registrada.

Um ponto importante é que a verificação de conflitos considera as solicitações pendentes e as reservas aprovadas. Dessa forma, o sistema evita que duas pessoas solicitem o mesmo recurso para períodos conflitantes.

A classe `ServicoReserva` fica responsável por essa verificação, mantendo as regras de negócio organizadas em um único lugar.

5.7. Considerações sobre o Diagrama de Sequência

A elaboração do Diagrama de Sequência permitiu compreender como as classes e os objetos interagem durante uma operação do sistema.

Foi possível representar desde o momento em que o usuário inicia uma solicitação até o registro da reserva ou a apresentação de uma mensagem de indisponibilidade.

A utilização de condições alternativas também permitiu representar diferentes situações que podem ocorrer durante o processo, como a existência de conflitos de horários ou a desativação de um recurso.

Dessa maneira, o diagrama complementa o Diagrama de Classes, demonstrando como as responsabilidades definidas anteriormente serão utilizadas durante a execução das funcionalidades.

5.8. Checklist da etapa

Diagrama de Sequência

Concluído

- Identificação dos participantes.
- Representação das mensagens.
- Definição da ordem de execução.
- Representação das condições alternativas.
- Verificação de disponibilidade.
- Verificação de conflitos.
- Registro da solicitação.
- Relação com o Diagrama de Classes.

5.9. Próxima etapa: Diagrama de Atividades

Na próxima etapa, vamos desenvolver o Diagrama de Atividades, que representará o fluxo completo de uma solicitação de reserva.

Ele demonstrará as decisões tomadas durante o processo, desde o acesso ao sistema até a aprovação ou rejeição pelo administrador.

Diferentemente do Diagrama de Sequência, que mostra a comunicação entre os objetos, o Diagrama de Atividades dará ênfase às etapas, decisões e caminhos possíveis do processo.

Assim, teremos uma visão mais completa do funcionamento do sistema antes de avançarmos para a análise dos princípios de orientação a objetos.

6. Diagrama de Atividades

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, vamos desenvolver o Diagrama de Atividades, utilizando a UML (Unified Modeling Language) para representar o fluxo de funcionamento de um processo dentro do sistema.

O Diagrama de Atividades permite visualizar as etapas de uma operação, demonstrando a sequência das ações, as decisões tomadas e os possíveis caminhos que o processo pode seguir.

Enquanto o Diagrama de Sequência demonstra a comunicação entre os objetos, o Diagrama de Atividades apresenta o funcionamento do processo como um todo.

Para este projeto, vamos representar o fluxo completo de uma solicitação de reserva, desde o acesso do usuário ao sistema até a aprovação ou rejeição pelo administrador.

6.1. Identificação das atividades

O processo foi dividido nas seguintes atividades:

Etapa	Atividade	Responsável
1	Acessar o sistema	Professor/Funcionário
2	Realizar login	Usuário e Sistema
3	Consultar recursos disponíveis	Professor/Funcionário
4	Selecionar laboratório ou equipamento	Professor/Funcionário
5	Informar data, horário e finalidade	Professor/Funcionário
6	Verificar situação do recurso	Sistema
7	Verificar conflitos de horário	Sistema
8	Registrar solicitação	Sistema
9	Consultar solicitações pendentes	Administrador
10	Analisar solicitação	Administrador
11	Aprovar ou rejeitar solicitação	Administrador
12	Atualizar situação da solicitação	Sistema

6.2. Descrição do funcionamento

Inicialmente, o professor ou funcionário acessa o sistema e informa suas credenciais para realizar o login.

Caso os dados estejam incorretos, o sistema apresenta uma mensagem de erro e solicita uma nova tentativa de acesso.

Após a autenticação, o usuário consulta os recursos disponíveis, seleciona um laboratório ou equipamento e informa os dados necessários para realizar a solicitação.

O sistema verifica se o recurso está ativo e se existe alguma solicitação pendente ou reserva aprovada para o mesmo período.

Se houver algum problema, o sistema informa o motivo e permite que o usuário escolha outro recurso ou horário.

Caso todas as verificações sejam realizadas com sucesso, a solicitação é registrada com o status Pendente.

Posteriormente, o administrador acessa a lista de solicitações pendentes, analisa as informações e decide se a solicitação será aprovada ou rejeitada.

Se aprovada, a reserva recebe o status Aprovada. Caso seja rejeitada, o administrador deverá informar o motivo da rejeição.

Por fim, o sistema atualiza a situação da solicitação, permitindo que o usuário consulte o resultado.

6.3. Diagrama de Atividades – Processo de Reserva

O diagrama abaixo apresenta o fluxo completo do processo, incluindo as verificações, decisões e caminhos alternativos.

6.4. Representação gráfica do processo

#chatgpt-mermaid-_r_175_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(237, 237, 237);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_175_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_175_ .edge-animation-

fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_175_ .error-icon{fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_175_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_175_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_175_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_175_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_175_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_175_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_175_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_175_ .marker.cross{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_175_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_175_ p{margin:0;}#chatgpt-mermaid-_r_175_ .label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ .cluster-label text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ .cluster-label span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ .cluster-label span p{background-color:transparent;}#chatgpt-mermaid-_r_175_ .label text,#chatgpt-mermaid-_r_175_ span{fill:rgb(237, 237, 237);color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ .node rect,#chatgpt-mermaid-_r_175_ .node circle,#chatgpt-mermaid-_r_175_ .node ellipse,#chatgpt-mermaid-_r_175_ .node polygon,#chatgpt-mermaid-_r_175_ .node path{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);stroke-width:1px;}#chatgpt-mermaid-_r_175_ .rough-node .label text,#chatgpt-mermaid-_r_175_ .node .label text,#chatgpt-mermaid-_r_175_ .image-shape .label,#chatgpt-mermaid-_r_175_ .icon-shape .label{text-anchor:middle;}#chatgpt-mermaid-_r_175_ .node .katex path{fill:#000;stroke:#000;stroke-width:1px;}#chatgpt-mermaid-_r_175_ .rough-node .label,#chatgpt-mermaid-_r_175_ .node .label,#chatgpt-mermaid-_r_175_ .image-shape .label,#chatgpt-mermaid-_r_175_ .icon-shape .label{text-align:center;}#chatgpt-mermaid-_r_175_ .node.clickable{cursor:pointer;}#chatgpt-mermaid-_r_175_ .root .anchor path{fill:rgb(175, 175, 175)!important;stroke-width:0;stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_175_ .arrowheadPath{fill:rgb(175, 175, 175);}#chatgpt-mermaid-_r_175_ .edgePath .path{stroke:rgb(175, 175, 175);stroke-width:1px;}#chatgpt-mermaid-_r_175_ .flowchart-link{stroke:rgb(175, 175, 175);fill:none;}#chatgpt-mermaid-_r_175_ .edgeLabel{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_175_ .edgeLabel p{background-color:rgb(0, 0, 0);}#chatgpt-mermaid-_r_175_ .edgeLabel rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_175_ .labelBkg{background-color:rgba(0, 0, 0, 0.5);}#chatgpt-mermaid-_r_175_ .cluster rect{fill:rgb(48, 48, 48);stroke:rgba(255, 255, 255, 0.15);stroke-width:1px;}#chatgpt-mermaid-_r_175_ .cluster text{fill:rgb(237, 237, 237);}#chatgpt-

mermaid-_r_175_ .cluster span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ div.mermaidTooltip{position:absolute;text-align:center;max-width:200px;padding:2px;font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:12px;background:rgb(48, 48, 48);border:1px solid rgba(255, 255, 255, 0.15);border-radius:2px;pointer-events:none;z-index:100;}#chatgpt-mermaid-_r_175_ .flowchartTitleText{text-anchor:middle;font-size:18px;fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_175_ rect.text{fill:none;stroke-width:0;}#chatgpt-mermaid-_r_175_ .icon-shape,#chatgpt-mermaid-_r_175_ .image-shape{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_175_ .icon-shape p,#chatgpt-mermaid-_r_175_ .image-shape p{background-color:rgb(0, 0, 0);padding:2px;}#chatgpt-mermaid-_r_175_ .icon-shape .label rect,#chatgpt-mermaid-_r_175_ .image-shape .label rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_175_ .label-icon{display:inline-block;height:1em;overflow:visible;vertical-align:-0.125em;}#chatgpt-mermaid-_r_175_ .node .label-icon path{fill:currentColor;stroke:revert;stroke-width:revert;}#chatgpt-mermaid-_r_175_ .node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_175_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_175_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_175_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_175_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_175_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_175_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_175_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_175_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_175_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_175_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_175_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_175_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_175_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_175_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_175_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_175_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_175_ .node text{font-size:14px;font-weight:600;letter-spacing:normal;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_175_ .edgeLabels text{font-size:13px;font-weight:600;letter-spacing:-0.08px;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_175_ .node tspan[font-weight="normal"],#chatgpt-mermaid-_r_175_ .edgeLabels tspan[font-weight="normal"]{font-weight:600;}#chatgpt-mermaid-_r_175_ .edgeLabel .label rect{opacity:1;rx:13px;ry:13px;fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-width:1px;}#chatgpt-mermaid-_r_175_ .node rect,#chatgpt-mermaid-_r_175_ .node circle,#chatgpt-mermaid-_r_175_ .node ellipse,#chatgpt-mermaid-_r_175_ .node polygon,#chatgpt-mermaid-_r_175_ .node path{fill:rgb(0, 40, 77);stroke:rgba(255,

255, 255, 0.1);stroke-width:1px;}#chatgpt-mermaid-_r_175_ .node rect{rx:16px;ry:16px;}#chatgpt-mermaid-_r_175_ .mermaid-decision .label-container{fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-dasharray:2,2;}#chatgpt-mermaid-_r_175_ .edgePaths .flowchart-link{stroke:rgb(175, 175, 175);stroke-width:1px;stroke-linecap:round;stroke-linejoin:round;}#chatgpt-mermaid-_r_175_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_175_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}InícioUsuário acessa o sistemaInforma e-mail e senhaDados de acesso válidos?Sistema apresenta mensagemde erroUsuário consulta os recursosSeleciona laboratório ouequipamentoInforma data, horário efinalidadeInformações completas?Sistema solicita correção dosdadosRecurso está ativo?Sistema informa que o recursoestá indisponívelExiste conflito de horário?Sistema informa o conflitoUsuário escolhe outro horárioou recursoSistema registra a solicitaçãoStatus: PENDENTEAdministrador consultasolicitações pendentesAdministrador analisa asolicitaçãoSolicitação aprovada?Sistema registra a aprovaçãoAdministrador informa omotivo da rejeiçãoSistema registra a rejeiçãoUsuário consulta o resultadoFimNãoSimNãoSimNãoSimSimNãoSimNão
Figura 4 – Diagrama de Atividades do processo de reserva de laboratórios e equipamentos. Fonte: Elaborado pelo autor (2026).

6.5. Explicação do diagrama

O fluxo apresentado pode ser dividido em quatro momentos principais:

1. Acesso ao sistema

O usuário acessa a aplicação e informa seu e-mail e senha. O sistema verifica os dados. Caso estejam incorretos, apresenta uma mensagem de erro e permite uma nova tentativa.

2. Solicitação da reserva

Após o acesso, o usuário consulta os recursos, seleciona aquele que deseja utilizar e informa os dados da solicitação.

O sistema verifica se as informações estão completas, se o recurso está ativo e se existe algum conflito de horário.

Caso alguma condição não seja atendida, o usuário deverá corrigir as informações ou escolher outro recurso ou horário.

3. Análise da solicitação

Quando todas as verificações são concluídas, o sistema registra a solicitação com o status Pendente.

O administrador consulta as solicitações pendentes e analisa as informações para decidir se a reserva será aprovada ou rejeitada.

4. Finalização

Se a solicitação for aprovada, o sistema atualiza seu status para Aprovada.

Se for rejeitada, o administrador informa o motivo e o sistema registra o status Rejeitada.

Em ambos os casos, o usuário poderá consultar o resultado da solicitação.

6.6. Elementos UML utilizados

O diagrama utiliza alguns elementos importantes da UML:

Elemento	Significado
Círculo preenchido	Indica o início do processo.
Retângulo arredondado	Representa uma atividade ou ação.
Losango	Representa uma decisão ou condição.
Setas	Indicam a sequência das atividades.
Círculo com contorno externo	Indica o encerramento do processo.

Os losangos representam decisões que podem alterar o caminho da execução. Por exemplo, quando o sistema verifica se existe conflito de horário, existem duas possibilidades: continuar com a solicitação ou retornar para que o usuário escolha outro período.

6.7. Relação com os outros diagramas

O Diagrama de Atividades complementa os modelos desenvolvidos anteriormente:

- Diagrama de Casos de Uso: identificou a funcionalidade de solicitar reserva e os usuários envolvidos.
- Diagrama de Classes: definiu as classes responsáveis pelas informações e operações do sistema.
- Diagrama de Sequência: demonstrou a comunicação entre os objetos durante uma solicitação.
- Diagrama de Atividades: apresentou o fluxo completo do processo, incluindo as decisões e os caminhos alternativos.

Os quatro diagramas representam diferentes perspectivas de uma mesma solução e devem permanecer coerentes entre si.

6.8. Considerações sobre o Diagrama de Atividades

A elaboração do Diagrama de Atividades permitiu representar de maneira organizada as etapas necessárias para realizar uma reserva.

Foi possível identificar os pontos em que o sistema precisa tomar decisões, como a validação dos dados, a verificação da disponibilidade dos recursos e a aprovação ou rejeição das solicitações.

Também foram considerados caminhos alternativos para situações em que as informações estejam incorretas ou o recurso não esteja disponível.

Dessa forma, o diagrama contribui para uma melhor compreensão do funcionamento do sistema e serve como referência para uma futura implementação.

6.9. Checklist da etapa

Diagrama de Atividades

Concluído

- Identificação das atividades.
- Representação do início e encerramento.
- Representação das decisões.
- Verificação de autenticação.
- Verificação de disponibilidade.
- Verificação de conflitos.
- Representação da aprovação e rejeição.

- Identificação dos caminhos alternativos.
- Relação com os demais diagramas UML.

6.10. Próxima etapa: Orientação a Objetos

KULeuvenX UMLx Certificate | edX

Análise dos princípios de orientação a objetos

Vamos explicar como abstração, encapsulamento, herança, polimorfismo, coesão e acoplamento foram aplicados na estrutura do sistema.

Essa próxima etapa será baseada diretamente no Diagrama de Classes que desenvolvemos, sem necessidade de criar uma arquitetura mais complexa.

Objetivo: demonstrar que você compreendeu como organizar as responsabilidades das classes e por que essas decisões ajudam a tornar o software mais fácil de entender, manter e evoluir.

7. Aplicação dos Princípios de Orientação a Objetos

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, serão apresentados os principais conceitos da Programação Orientada a Objetos (POO) e como eles podem ser aplicados no Sistema de Reserva de Laboratórios e Equipamentos.

A Programação Orientada a Objetos é um paradigma de desenvolvimento que organiza o sistema em classes e objetos, permitindo representar elementos do mundo real dentro de uma aplicação.

No projeto, por exemplo, podemos representar usuários, administradores, laboratórios, equipamentos e reservas por meio de classes.

A utilização desses conceitos ajuda a organizar o código, evitar repetições desnecessárias e facilitar futuras alterações no sistema.

7.1. Abstração

A abstração consiste em identificar as características e funcionalidades mais importantes de um elemento, representando apenas aquilo que é necessário para o funcionamento do sistema.

No projeto, podemos observar esse conceito na criação da classe `Usuario`.

Essa classe representa características comuns entre os diferentes tipos de usuários, como:

- Identificador;
- Nome;
- E-mail;
- Senha;
- Situação do cadastro.

A partir dela, são especializadas as classes `Professor`, `Funcionario` e `Administrador`.

Da mesma forma, a classe `Recurso` representa características comuns aos laboratórios e equipamentos, como identificador, nome, descrição e situação de funcionamento.

Assim, não é necessário repetir todas essas informações nas classes especializadas.

Aplicação no projeto: as classes abstratas `Usuario` e `Recurso` permitem representar características compartilhadas, enquanto as classes derivadas apresentam informações específicas de cada tipo.

7.2. Encapsulamento

O encapsulamento consiste em proteger os dados de um objeto, evitando que sejam modificados diretamente de maneira inadequada.

Para isso, os atributos de uma classe podem ser definidos como privados (`private`), permitindo que seu acesso e alteração sejam controlados por métodos específicos.

Por exemplo, na classe `Reserva`, os atributos podem ser:

```
public class Reserva {
    private int id;
```

```
    private String data;
    private String horaInicio;
    private String horaFim;
    private String finalidade;
    private StatusReserva status;
}
```

Nesse exemplo, os atributos não podem ser acessados diretamente por outras classes.

Caso seja necessário consultar ou modificar alguma informação, podem ser utilizados métodos específicos, como:

```
public StatusReserva getStatus() {
    return status;
}

public void cancelar() {
    if (status == StatusReserva.PENDENTE ||
        status == StatusReserva.APROVADA) {
        status = StatusReserva.CANCELADA;
    }
}
```

O método `cancelar()` demonstra como uma alteração pode ser controlada por uma regra definida pela própria classe.

Aplicação no projeto: o encapsulamento ajuda a proteger as informações das reservas e evita alterações indevidas nos seus estados.

7.3. Herança

A herança permite que uma classe aproveite atributos e métodos de outra classe, evitando a repetição de código.

No projeto, a classe `Usuario` funciona como uma classe mais geral, enquanto `Professor`, `Funcionario` e `Administrador` representam tipos específicos de usuários.

A estrutura pode ser representada da seguinte maneira:

Usuario

id, nome, email, senhaHash, ativo

Herança

Professor

matricula

Funcionario

matricula, setor

Administrador

nivelAcesso

Outro exemplo é a classe `Recurso`, que pode ser especializada em `Laboratorio` e `Equipamento`.

Aplicação no projeto: a herança permite compartilhar características comuns entre os diferentes tipos de usuários e recursos, deixando a estrutura mais organizada e reduzindo a duplicação de informações.

7.4. Polimorfismo

O polimorfismo permite que diferentes classes utilizem uma mesma operação, apresentando comportamentos específicos de acordo com o tipo do objeto.

Por exemplo, podemos imaginar que todos os recursos possuem um método chamado `consultarDetalhes()`.

Entretanto, um laboratório possui características como capacidade e localização, enquanto um equipamento possui patrimônio e estado de conservação.

Em Java, isso poderia ser representado assim:

```
public abstract class Recurso {

    public abstract void consultarDetalhes();

}
```

Na classe `Laboratorio`:

```
public class Laboratorio extends Recurso {

    @Override
    public void consultarDetalhes() {
```

```
        System.out.println("Exibindo capacidade e localização.");
    }
}
```

Na classe `Equipamento`:

```
public class Equipamento extends Recurso {

    @Override
    public void consultarDetalhes() {
        System.out.println("Exibindo patrimônio e estado de
conservação.");
    }
}
```

Dessa forma, podemos chamar o mesmo método para diferentes tipos de recursos:

```
Recurso recurso1 = new Laboratorio();
Recurso recurso2 = new Equipamento();

recurso1.consultarDetalhes();
recurso2.consultarDetalhes();
```

Apesar de utilizarem o mesmo método, cada objeto executa seu próprio comportamento.

Aplicação no projeto: o polimorfismo permite que o sistema trabalhe com diferentes tipos de recursos utilizando uma estrutura comum, sem precisar tratar cada tipo de maneira completamente independente.

Observação: o método `consultarDetalhes()` é uma proposta de exemplo para demonstrar o conceito. Ele não precisa ser acrescentado ao diagrama de classes atual se não fizer parte das funcionalidades planejadas.

7.5. Coesão

A coesão representa o quanto as responsabilidades de uma classe estão relacionadas entre si.

Uma classe possui boa coesão quando seus métodos e atributos estão relacionados a uma finalidade específica.

No projeto, podemos observar esse conceito na classe `Reserva`, responsável por representar uma solicitação de utilização de um recurso.

Ela possui informações como data, horário, finalidade e situação da reserva, além de operações relacionadas ao seu ciclo de vida, como aprovar, rejeitar e cancelar.

Por outro lado, seria inadequado colocar nessa mesma classe funcionalidades como cadastrar usuários, gerenciar equipamentos e controlar o acesso dos administradores, pois essas responsabilidades pertencem a outras partes do sistema.

Aplicação no projeto: cada classe deve possuir responsabilidades bem definidas, facilitando a compreensão e a manutenção do sistema.

7.6. Acoplamento

O acoplamento representa o nível de dependência entre as classes de um sistema.

Quando uma classe depende excessivamente de outras, qualquer alteração pode provocar modificações em diferentes partes da aplicação.

Por isso, é interessante manter um baixo acoplamento, permitindo que as classes tenham maior independência.

No projeto, a classe `ServicoReserva` é responsável por coordenar operações relacionadas às solicitações, como verificar conflitos, registrar reservas e processar aprovações e rejeições.

Dessa maneira, as regras de reserva não precisam ficar concentradas em uma única classe ou misturadas com as responsabilidades de apresentação da aplicação.

Aplicação no projeto: separar as responsabilidades entre `Reserva`, `Recurso`, `Usuario` e `ServicoReserva` contribui para diminuir dependências desnecessárias e facilitar futuras alterações.

7.7. Resumo dos conceitos aplicados

Conceito	Aplicação no projeto	Benefício
Abstração	Classes `Usuario` e `Recurso`	Representa características essenciais.

Encapsulamento	Atributos privados e métodos controlados	Protege os dados.
Herança	Professor, Funcionário e Administrador derivam de Usuario	Reaproveita características.
Polimorfismo	Diferentes recursos implementam comportamentos próprios	Permite flexibilidade.
Coesão	Cada classe possui responsabilidades específicas	Facilita a manutenção.
Acoplamento	Separação das responsabilidades entre classes e serviços	Reduz dependências.

7.8. Considerações sobre a aplicação da Orientação a Objetos

A aplicação dos princípios de Orientação a Objetos permite organizar melhor a estrutura do Sistema de Reserva de Laboratórios e Equipamentos.

Com a abstração e a herança, é possível representar diferentes tipos de usuários e recursos sem repetir desnecessariamente suas características.

O encapsulamento contribui para proteger as informações e controlar as alterações realizadas nos objetos. Já o polimorfismo permite que diferentes classes apresentem comportamentos específicos por meio de operações comuns.

Além disso, a preocupação com coesão e acoplamento ajuda a distribuir as responsabilidades de maneira adequada, tornando o sistema mais organizado e facilitando sua manutenção.

Dessa forma, os conceitos de Orientação a Objetos contribuem para o desenvolvimento de uma solução que poderá receber novas funcionalidades no futuro sem exigir alterações desnecessárias em toda a estrutura.

7.9. Checklist da etapa

Orientação a Objetos

Concluído

- Explicação da abstração.
- Aplicação do encapsulamento.
- Aplicação da herança.
- Demonstração do polimorfismo.
- Explicação da coesão.
- Explicação do acoplamento.
- Relação dos conceitos com as classes do projeto.

Próxima etapa: Padrões de Projeto (Design Patterns)

Na próxima etapa, vamos selecionar pelo menos dois padrões de projeto que façam sentido para o sistema, explicar o problema que cada um resolve, como seria sua aplicação e quais são seus benefícios e limitações.

A ideia será escolher padrões que realmente contribuam para o projeto, evitando adicionar complexidade sem necessidade.

8. Padrões de Projeto (Design Patterns)

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Os padrões de projeto, também conhecidos como Design Patterns, são soluções reutilizáveis para problemas comuns encontrados durante o desenvolvimento de software.

Eles não são códigos prontos que devem ser utilizados em qualquer aplicação. São modelos que ajudam os desenvolvedores a organizar melhor as classes, distribuir responsabilidades e resolver determinados problemas de implementação.

No Sistema de Reserva de Laboratórios e Equipamentos, foram escolhidos dois padrões de projeto: Factory Method e Observer.

A escolha considera as necessidades atuais do sistema e a possibilidade de adicionar novas funcionalidades no futuro, sem tornar a estrutura desnecessariamente complexa.

8.1. Padrão Factory Method (Método de Fábrica)

Factory Method Pattern Diagram 490x250

Categoria: Criacional

Objetivo: organizar a criação de objetos, deixando a responsabilidade de instanciar determinados tipos de objetos para classes específicas.

Problema identificado

No sistema, existem diferentes tipos de recursos que podem ser cadastrados, como laboratórios e equipamentos.

Cada tipo possui características próprias. Um laboratório, por exemplo, possui capacidade e localização, enquanto um equipamento possui número de patrimônio e estado de conservação.

Se a criação desses objetos ficar concentrada em uma única parte do sistema, pode ser necessário adicionar várias condições para verificar qual tipo de recurso deve ser criado.

Com o crescimento da aplicação, esse processo pode ficar mais difícil de manter.

Escolha do padrão

O padrão Factory Method foi escolhido porque permite separar a criação dos diferentes tipos de recursos da lógica principal do sistema.

Assim, cada tipo de criador fica responsável por instanciar um tipo específico de recurso.

Aplicação no projeto

A estrutura poderia funcionar da seguinte maneira:

- `Recurso`: classe abstrata que representa os recursos do sistema.
- `Laboratorio`: classe que representa os laboratórios.
- `Equipamento`: classe que representa os equipamentos.
- `CriadorRecurso`: define o método responsável pela criação.
- `CriadorLaboratorio`: realiza a criação de laboratórios.
- `CriadorEquipamento`: realiza a criação de equipamentos.

Exemplo simplificado em Java:

```
abstract class Recurso {
    protected String nome;

    public Recurso(String nome) {
```

```
        this.nome = nome;
    }
}

class Laboratorio extends Recurso {
    public Laboratorio(String nome) {
        super(nome);
    }
}

class Equipamento extends Recurso {
    public Equipamento(String nome) {
        super(nome);
    }
}
```

A criação dos objetos ficaria separada em classes específicas:

```
abstract class CriadorRecurso {
    public abstract Recurso criarRecurso(String nome);
}

class CriadorLaboratorio extends CriadorRecurso {
    @Override
    public Recurso criarRecurso(String nome) {
        return new Laboratorio(nome);
    }
}

class CriadorEquipamento extends CriadorRecurso {
    @Override
    public Recurso criarRecurso(String nome) {
        return new Equipamento(nome);
    }
}
```

Dessa forma, quando o administrador precisar cadastrar um laboratório, o sistema utilizará o criador de laboratórios. Para cadastrar um equipamento, utilizará o criador de equipamentos.

Benefícios

- Organiza a criação dos diferentes tipos de recursos.

- Separa a responsabilidade de criação das demais funcionalidades.
- Facilita a inclusão de novos tipos de recursos.
- Reduz a necessidade de concentrar toda a lógica de criação em uma única classe.

Limitações

A utilização desse padrão acrescenta classes à estrutura do sistema. Para uma aplicação muito pequena, essa separação pode representar uma complexidade desnecessária.

Por isso, sua utilização é mais interessante quando a criação de diferentes tipos de recursos realmente precisa ser organizada ou quando existe a previsão de expansão dessa funcionalidade.

8.2. Padrão Observer (Observador)

Observer Design Pattern. Introduction | by Deva25 | Medium

Categoria: Comportamental

Objetivo: permitir que diferentes partes do sistema sejam notificadas automaticamente quando determinado objeto sofre uma alteração.

Problema identificado

No sistema, uma solicitação de reserva pode passar por diferentes situações:

- Pendente;
- Aprovada;
- Rejeitada;
- Cancelada.

Quando o administrador aprova ou rejeita uma solicitação, o usuário que realizou o pedido precisa ter acesso à atualização.

Uma possibilidade seria fazer a própria classe `Reserva` controlar diretamente todos os meios de comunicação com os usuários. Entretanto, isso misturaria a responsabilidade de administrar o estado da reserva com a responsabilidade de enviar notificações.

Escolha do padrão

O padrão Observer foi escolhido porque permite que o sistema informe automaticamente as partes interessadas quando ocorre uma alteração no estado de uma reserva.

Dessa maneira, a classe responsável pela reserva não precisa conhecer todos os detalhes de como as notificações serão enviadas.

Aplicação no projeto

A implementação poderia funcionar da seguinte maneira:

1. O usuário solicita uma reserva.
2. O sistema registra a solicitação como Pendente.
3. O administrador analisa o pedido.
4. O administrador aprova ou rejeita a solicitação.
5. O sistema atualiza o status da reserva.
6. Os observadores cadastrados recebem uma notificação sobre a alteração.

A estrutura poderia possuir as seguintes classes e interfaces:

- `Reserva`: representa a solicitação e informa quando seu estado é alterado.
- `ObservadorReserva`: define o contrato para receber notificações.
- `NotificadorUsuario`: implementa o recebimento e encaminhamento das notificações.
- `ServicoReserva`: coordena as operações de aprovação e rejeição.

Exemplo simplificado em Java:

```
interface ObservadorReserva {
    void notificar(String mensagem);
}

class NotificadorUsuario implements ObservadorReserva {

    @Override
    public void notificar(String mensagem) {
        System.out.println("Notificação: " + mensagem);
    }
}
```

A classe `Reserva` poderia manter os observadores e avisá-los quando o status fosse alterado:

```
import java.util.ArrayList;
import java.util.List;

class Reserva {

    private String status = "PENDENTE";

    private List<ObservadorReserva> observadores =
            new ArrayList<>();

    public void adicionarObservador(ObservadorReserva observador) {
        observadores.add(observador);
    }

    private void notificarObservadores() {
        for (ObservadorReserva observador : observadores) {
            observador.notificar(
                "A situação da reserva foi alterada para: " + status
            );
        }
    }

    public void aprovar() {
        status = "APROVADA";
        notificarObservadores();
    }

    public void rejeitar() {
        status = "REJEITADA";
        notificarObservadores();
    }
}
```

Nesse exemplo, quando a reserva é aprovada ou rejeitada, os observadores cadastrados recebem uma mensagem informando a alteração.

O exemplo utiliza mensagens no console apenas para demonstrar o funcionamento do padrão. Em uma futura implementação, as notificações poderiam ser exibidas dentro do sistema ou enviadas por e-mail.

Benefícios

- Permite comunicar alterações automaticamente.

- Separa o controle das reservas do envio de notificações.
- Facilita a inclusão de novos meios de comunicação.
- Reduz a dependência direta entre a classe `Reserva` e os mecanismos de notificação.

Limitações

A utilização do Observer exige o gerenciamento dos observadores cadastrados. Caso esse controle não seja realizado corretamente, podem ocorrer notificações duplicadas ou desnecessárias.

Além disso, em uma aplicação maior, seria necessário considerar falhas no envio das mensagens e mecanismos para tentar realizar o envio novamente.

8.3. Comparação dos padrões escolhidos

Característica	Factory Method	Observer
Categoria	Criacional	Comportamental
Principal finalidade	Organizar a criação de objetos	Notificar alterações
Aplicação	Criação de laboratórios e equipamentos	Atualização do status das reservas
Principal benefício	Separação da responsabilidade de criação	Comunicação entre objetos
Limitação	Aumenta o número de classes	Exige gerenciamento dos observadores

8.4. Relação com os princípios de Orientação a Objetos

Os padrões escolhidos também se relacionam com os conceitos de Orientação a Objetos apresentados na etapa anterior.

Factory Method:

- Abstração: utiliza a classe abstrata `Recurso`.
- Encapsulamento: concentra a criação dos objetos em classes específicas.
- Polimorfismo: permite trabalhar com diferentes tipos de recursos por meio da classe `Recurso`.
- Baixo acoplamento: separa a criação dos objetos das funcionalidades que utilizam esses recursos.

Observer:

- Abstração: utiliza a interface `ObservadorReserva`.
- Encapsulamento: mantém a lógica de alteração e notificação organizada.
- Baixo acoplamento: permite que a reserva comunique alterações sem depender diretamente de cada mecanismo de notificação.

8.5. Considerações sobre os padrões de projeto

A escolha dos padrões Factory Method e Observer considera duas necessidades do Sistema de Reserva de Laboratórios e Equipamentos.

O Factory Method permite organizar a criação de diferentes tipos de recursos, como laboratórios e equipamentos. Já o Observer permite comunicar automaticamente as alterações no estado das reservas aos usuários interessados.

Os dois padrões podem contribuir para uma estrutura mais organizada e facilitar futuras modificações no sistema.

Entretanto, é importante destacar que os padrões de projeto não devem ser utilizados apenas para aumentar a quantidade de classes ou tornar o código aparentemente mais sofisticado. Sua aplicação deve estar relacionada a problemas reais ou a necessidades futuras justificadas.

Dessa forma, a utilização desses padrões é proposta como uma possibilidade para a implementação futura do sistema, não sendo necessário desenvolver todas essas estruturas nesta atividade.

8.6. Checklist da etapa

Padrões de Projeto

Concluído

- Seleção de dois padrões de projeto.
- Identificação dos problemas que eles resolvem.

- Justificativa das escolhas.
- Explicação da aplicação no sistema.
- Exemplos simplificados em Java.
- Apresentação dos benefícios.
- Identificação das limitações.
- Relação com os princípios de Orientação a Objetos.

Próxima etapa: Metodologia de Desenvolvimento

Na próxima etapa, vamos comparar metodologias de desenvolvimento de software, como Cascata e Scrum, e justificar qual delas seria mais adequada para organizar o desenvolvimento do Sistema de Reserva de Laboratórios e Equipamentos.

9. Metodologia de Desenvolvimento de Software

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

A metodologia de desenvolvimento de software define como as atividades de um projeto serão organizadas, acompanhadas e executadas.

Sua escolha é importante porque influencia a comunicação entre os envolvidos, a divisão das tarefas, o acompanhamento do progresso e a capacidade de realizar alterações durante o desenvolvimento.

Para o Sistema de Reserva de Laboratórios e Equipamentos, serão comparadas duas abordagens: Modelo Cascata (Waterfall) e Scrum.

Após a comparação, será apresentada a metodologia considerada mais adequada para o projeto.

9.1. Modelo Cascata (Waterfall)

What Is a Systems Development Life Cycle (SDLC)?
wallarm.com

O Modelo Cascata é uma abordagem tradicional de desenvolvimento de software, na qual as etapas são organizadas de maneira sequencial.

Isso significa que uma fase precisa ser concluída antes que a próxima seja iniciada.

As principais etapas são:

1. Levantamento de requisitos: identificação das necessidades dos usuários e definição das funcionalidades.
2. Análise e projeto: organização da estrutura do sistema, classes, diagramas e arquitetura.
3. Implementação: desenvolvimento do código e das funcionalidades.
4. Testes: verificação do funcionamento e identificação de erros.
5. Implantação: disponibilização do sistema para utilização.
6. Manutenção: correção de problemas e realização de melhorias.

Vantagens

- Possui etapas bem definidas.
- Facilita o planejamento inicial.
- Permite documentar cada fase do desenvolvimento.
- Pode funcionar bem quando os requisitos são estáveis e conhecidos desde o início.

Desvantagens

- Alterações nos requisitos podem gerar retrabalho.
- O usuário geralmente participa menos durante as etapas de implementação.
- Problemas importantes podem ser identificados somente em fases posteriores.
- A entrega das funcionalidades costuma ocorrer mais ao final do projeto.

9.2. Scrum

Scrum Introduction: Roles, Ceremonies, and Processes for Agile Teams
scalablepath.com

O Scrum é um framework ágil utilizado para organizar o desenvolvimento de produtos por meio de ciclos de trabalho chamados Sprints.

Em vez de desenvolver todo o sistema de uma única vez, as funcionalidades são divididas em partes menores, permitindo entregas incrementais e avaliações frequentes.

No Scrum, existem três responsabilidades principais:

- Product Owner: representa os interesses relacionados ao produto, organiza as prioridades e mantém o Product Backlog.
- Scrum Master: ajuda a equipe a compreender e aplicar o Scrum, além de auxiliar na remoção de impedimentos.

- Developers: profissionais responsáveis por desenvolver o incremento do produto.

O trabalho é organizado por meio dos seguintes elementos e eventos:

- Product Backlog: lista de funcionalidades, melhorias e necessidades do sistema.
- Sprint Planning: planejamento das atividades que serão realizadas na Sprint.
- Daily Scrum: encontro diário para acompanhar o progresso e ajustar o plano de trabalho.
- Sprint Review: apresentação e avaliação do resultado desenvolvido.
- Sprint Retrospective: momento para identificar melhorias na forma de trabalho.

Vantagens

- Permite desenvolver e entregar funcionalidades gradualmente.
- Facilita a identificação de problemas durante o desenvolvimento.
- Permite adaptar prioridades conforme novas necessidades.
- Incentiva a comunicação entre os integrantes da equipe.
- Possibilita receber avaliações frequentes dos usuários.

Desvantagens

- Exige participação e comunicação frequentes.
- Mudanças constantes de prioridade podem prejudicar o planejamento.
- Depende de uma boa organização do Product Backlog.
- Pode apresentar dificuldades quando os envolvidos não compreendem suas responsabilidades.

9.3. Comparação entre Cascata e Scrum

Critério	Cascata	Scrum
Organização	Etapas sequenciais	Ciclos de trabalho
Planejamento	Mais concentrado no início	Revisado a cada Sprint
Alterações	Podem gerar retrabalho	Podem ser incorporadas às próximas Sprints

Participação do usuário	Geralmente maior no início e nas validações	Avaliações frequentes
Entregas	Geralmente concentradas no final	Incrementais
Documentação	Planejada por etapas	Produzida conforme a necessidade
Flexibilidade	Menor	Maior

9.4. Metodologia escolhida para o projeto

Para o desenvolvimento do Sistema de Reserva de Laboratórios e Equipamentos, a metodologia escolhida é o Scrum.

Essa escolha considera que o sistema pode receber novas necessidades durante seu desenvolvimento, como melhorias na consulta de disponibilidade, alterações nas regras de aprovação e inclusão de novas funcionalidades.

Com o Scrum, seria possível desenvolver primeiro as funções essenciais e, posteriormente, adicionar melhorias conforme os resultados dos testes e as avaliações dos usuários.

Além disso, como o sistema envolve diferentes perfis de usuários, seria interessante apresentar versões parciais para professores, funcionários e administradores, permitindo verificar se as funcionalidades atendem às necessidades identificadas.

A escolha do Scrum não significa que a documentação deixa de ser importante. Os requisitos e diagramas desenvolvidos nas etapas anteriores continuam servindo como referência para orientar a equipe.

9.5. Organização das Sprints

O desenvolvimento poderia ser dividido em cinco Sprints, cada uma com duração sugerida de duas semanas.

Planejamento das Sprints

Estimativa inicial de desenvolvimento: 10 semanas.

1. Sprint 1 – Estrutura inicial e usuários
    - Preparação do ambiente de desenvolvimento.
    - Implementação do cadastro de usuários.
    - Autenticação e controle de acesso.
    - Definição dos perfis de usuário.

Semanas 1 e 2

2. Sprint 2 – Gerenciamento de recursos
    - Cadastro de laboratórios.
    - Cadastro de equipamentos.
    - Consulta e atualização dos recursos.
    - Ativação e desativação de recursos.

Semanas 3 e 4

3. Sprint 3 – Solicitação de reservas
    - Consulta de disponibilidade.
    - Seleção de recursos.
    - Solicitação de reservas.
    - Verificação de conflitos de horário.

Semanas 5 e 6

4. Sprint 4 – Aprovação e acompanhamento
    - Consulta de solicitações pendentes.
    - Aprovação e rejeição.
    - Registro do motivo da rejeição.
    - Consulta do histórico e cancelamento.

Semanas 7 e 8

5. Sprint 5 – Testes e melhorias
    - Testes das funcionalidades.
    - Correção de erros.
    - Verificação das permissões.
    - Avaliação da usabilidade.
    - Preparação para implantação.

Semanas 9 e 10

As durações são estimativas para planejamento acadêmico, não prazos comprovados. A quantidade de trabalho poderá ser ajustada conforme o tamanho da equipe e a complexidade encontrada durante a implementação.

9.6. Exemplo de aplicação do Scrum

Imagine que a equipe esteja trabalhando na terceira Sprint, responsável pela solicitação de reservas.

Durante o planejamento, são selecionadas as funcionalidades prioritárias:

- Consultar disponibilidade de recursos.
- Selecionar data e horário.
- Informar a finalidade da reserva.
- Verificar conflitos.
- Registrar a solicitação.

Durante os dias de desenvolvimento, a equipe acompanha o andamento das tarefas e identifica possíveis dificuldades.

Ao final da Sprint, apresenta a funcionalidade desenvolvida e verifica se o processo está funcionando conforme os requisitos definidos.

Caso seja identificado um problema, como a possibilidade de duas solicitações conflitantes serem registradas, a equipe poderá corrigir a situação e organizar as próximas atividades.

Esse processo permite identificar falhas antes que todo o sistema esteja concluído.

9.7. Considerações sobre a metodologia escolhida

A comparação entre o Modelo Cascata e o Scrum permitiu identificar diferentes formas de organizar o desenvolvimento de software.

O Modelo Cascata apresenta uma estrutura mais sequencial, sendo interessante para projetos em que os requisitos estão bem definidos e dificilmente serão alterados.

Já o Scrum permite trabalhar com entregas incrementais, avaliações frequentes e adaptações durante o desenvolvimento.

Para o Sistema de Reserva de Laboratórios e Equipamentos, o Scrum foi considerado mais adequado porque permite desenvolver as funcionalidades gradualmente, testar cada parte e realizar melhorias conforme as necessidades dos usuários.

Dessa maneira, a equipe poderá priorizar inicialmente as funcionalidades essenciais, como autenticação, cadastro de recursos e solicitação de reservas, deixando melhorias adicionais para etapas posteriores.

9.8. Checklist da etapa

Metodologia de Desenvolvimento

Concluído

- Explicação do Modelo Cascata.
- Explicação do Scrum.
- Identificação das vantagens e desvantagens.
- Comparação entre as metodologias.
- Justificativa da metodologia escolhida.
- Planejamento das Sprints.
- Exemplo de aplicação no projeto.

Próxima etapa: Qualidade, Riscos e Sustentabilidade

Na próxima etapa, vamos identificar os principais riscos do sistema, apresentar medidas para reduzir esses problemas e definir critérios de qualidade relacionados à segurança, confiabilidade, desempenho, usabilidade e manutenção.

Também vamos abordar a sustentabilidade do software, considerando seu custo de manutenção, aproveitamento dos recursos tecnológicos e possibilidade de evolução ao longo do tempo.

10. Qualidade, Riscos e Sustentabilidade

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, serão apresentados os principais aspectos relacionados à qualidade, aos riscos e à sustentabilidade do Sistema de Reserva de Laboratórios e Equipamentos.

Esses fatores são importantes porque não basta desenvolver um sistema que execute suas funcionalidades. Também é necessário garantir que ele seja seguro, confiável, fácil de utilizar e que possa continuar funcionando e recebendo melhorias ao longo do tempo.

Além disso, é importante identificar possíveis problemas antes da implementação, definindo medidas para evitar ou reduzir seus impactos.

10.1. Qualidade do software

A qualidade de software está relacionada à capacidade de um sistema atender às necessidades dos usuários e funcionar corretamente.

Para o projeto, foram definidos alguns critérios de qualidade que deverão ser considerados durante o desenvolvimento.

Critério	Aplicação no sistema
Funcionalidade	Permitir cadastrar recursos, consultar disponibilidade e solicitar reservas.
Confiabilidade	Evitar reservas duplicadas e manter as informações consistentes.
Usabilidade	Apresentar uma interface simples e fácil de compreender.
Desempenho	Responder às operações principais em até 3 segundos, em condições normais de uso.
Segurança	Proteger os dados dos usuários e restringir funcionalidades conforme os perfis de acesso.
Manutenibilidade	Organizar o código para facilitar correções e futuras alterações.
Compatibilidade	Permitir utilização nos principais navegadores atuais.

Esses critérios servem como referência para avaliar se o sistema está atendendo às expectativas definidas no projeto.

10.1.1. Critérios de qualidade e aceitação

Para verificar a qualidade do sistema, é necessário estabelecer condições que permitam avaliar o funcionamento das funcionalidades.

Alguns exemplos de critérios de aceitação são:

- Autenticação: o sistema deve permitir o acesso somente com credenciais válidas.
- Controle de acesso: professores e funcionários não podem acessar funcionalidades exclusivas do administrador.
- Disponibilidade: o sistema deve identificar quando um recurso já possui uma solicitação pendente ou reserva aprovada no mesmo período.
- Solicitação: uma reserva só pode ser registrada quando os dados obrigatórios estiverem preenchidos.
- Aprovação: somente o administrador pode aprovar ou rejeitar solicitações.
- Rejeição: o administrador deve informar um motivo para rejeitar uma solicitação.
- Histórico: o usuário deve conseguir consultar suas próprias solicitações e respectivas situações.
- Desempenho: as operações principais devem apresentar tempo de resposta de até 3 segundos em condições normais de uso.

Esses critérios poderão ser utilizados posteriormente durante os testes para verificar se cada funcionalidade está funcionando conforme o esperado.

10.2. Identificação dos riscos do projeto

Os riscos representam situações que podem prejudicar o desenvolvimento ou o funcionamento do sistema.

Identificá-los antecipadamente permite que a equipe prepare medidas para reduzir a possibilidade de ocorrência desses problemas.

Para este projeto, foram identificados os seguintes riscos:

Risco	Probabilidade	Impacto	Medida preventiva
Reservas duplicadas para o mesmo recurso e horário	Média	Alto	Implementar verificação de conflitos antes da confirmação.
Acesso indevido às informações	Média	Alto	Utilizar autenticação e controle de permissões.
Perda de informações	Baixa	Alto	Implementar cópias de segurança periódicas.

Falhas durante a aprovação de reservas	Média	Médio	Realizar testes nas operações de aprovação e rejeição.
Dificuldade de utilização do sistema	Média	Médio	Desenvolver uma interface simples e realizar testes de usabilidade.
Atrasos no desenvolvimento	Média	Médio	Organizar as atividades em Sprints e acompanhar o progresso.
Aumento da quantidade de usuários	Média	Médio	Planejar uma estrutura que permita melhorias de desempenho e capacidade.

As classificações de probabilidade e impacto representam estimativas iniciais para o cenário proposto, não resultados de uma avaliação realizada em uma instituição real.

10.3. Medidas para reduzir os riscos

Além de identificar os riscos, é necessário estabelecer ações que ajudem a evitar problemas ou diminuir suas consequências.

1. Prevenção de reservas duplicadas

O sistema deverá verificar se já existe uma solicitação pendente ou uma reserva aprovada para o mesmo recurso e período.

Essa verificação deverá considerar também possíveis solicitações realizadas simultaneamente, evitando que duas pessoas consigam reservar o mesmo recurso para horários conflitantes.

2. Proteção das informações

O acesso deverá ser realizado por meio de autenticação, com permissões específicas para cada perfil.

As senhas deverão ser armazenadas utilizando funções de hash apropriadas para senhas, como Argon2 ou bcrypt, evitando o armazenamento em texto puro.

3. Cópias de segurança

Deverão ser realizadas cópias de segurança periódicas das informações importantes, permitindo sua recuperação em caso de falhas ou perda de dados.

4. Testes das funcionalidades

As funcionalidades deverão ser testadas antes da disponibilização do sistema, principalmente aquelas relacionadas à criação, aprovação, rejeição e cancelamento de reservas.

5. Facilidade de utilização

A interface deverá apresentar informações claras, mensagens de orientação e confirmação das operações realizadas, reduzindo possíveis dificuldades para os usuários.

6. Organização do desenvolvimento

A utilização do Scrum permitirá acompanhar o andamento das atividades, identificar dificuldades e reorganizar prioridades quando necessário.

10.4. Sustentabilidade do software

A sustentabilidade de um software está relacionada à sua capacidade de continuar sendo utilizado, mantido e atualizado ao longo do tempo, considerando aspectos econômicos, ambientais, sociais e tecnológicos.

No projeto, foram identificadas quatro dimensões importantes.

10.4.1. Sustentabilidade econômica

A sustentabilidade econômica considera os custos envolvidos no desenvolvimento, na implantação e na manutenção do sistema.

Como a solução será utilizada por uma instituição de ensino, é importante evitar gastos desnecessários com ferramentas e tecnologias.

Algumas medidas propostas são:

- Priorizar ferramentas gratuitas ou de baixo custo quando forem suficientes.
- Evitar a contratação de serviços que não sejam necessários.
- Planejar os custos de hospedagem e manutenção.
- Reaproveitar componentes e funcionalidades existentes.
- Organizar o código para facilitar futuras atualizações.

Essas medidas podem ajudar a reduzir os custos de manutenção e tornar o sistema mais viável financeiramente.

10.4.2. Sustentabilidade ambiental

A sustentabilidade ambiental considera os impactos do uso de recursos tecnológicos e a possibilidade de reduzir desperdícios.

No sistema proposto, algumas medidas são:

- Substituir parte dos processos realizados em papel por operações digitais.
- Evitar impressões desnecessárias de solicitações e relatórios.
- Utilizar recursos computacionais de maneira eficiente.
- Evitar processamento desnecessário durante as consultas.
- Planejar a utilização adequada dos serviços de hospedagem.

Dessa maneira, o sistema poderá contribuir para a redução do consumo de papel e para um uso mais consciente dos recursos tecnológicos.

10.4.3. Sustentabilidade social

A sustentabilidade social considera como o sistema pode beneficiar as pessoas que irão utilizá-lo.

Nesse sentido, a solução deverá:

- Facilitar o acesso às informações sobre laboratórios e equipamentos.
- Oferecer uma interface simples e acessível.
- Permitir que professores e funcionários acompanhem suas solicitações.
- Reduzir conflitos e dificuldades na organização das reservas.
- Proteger os dados pessoais dos usuários.

Também deverá considerar boas práticas de acessibilidade, como textos legíveis, contraste adequado, navegação por teclado e identificação clara dos elementos da interface.

10.4.4. Sustentabilidade tecnológica

A sustentabilidade tecnológica está relacionada à capacidade de manter e evoluir o sistema sem depender de soluções difíceis de atualizar ou excessivamente complexas.

Algumas medidas propostas são:

- Utilizar tecnologias conhecidas e com documentação disponível.

- Organizar o código em componentes e módulos com responsabilidades definidas.
- Manter as dependências atualizadas.
- Registrar informações importantes sobre o funcionamento do sistema.
- Realizar testes após alterações.
- Evitar dependências desnecessárias de ferramentas específicas.

A separação de responsabilidades e os princípios de Orientação a Objetos apresentados anteriormente também contribuem para facilitar a manutenção e a evolução da aplicação.

10.5. Relação entre qualidade, riscos e sustentabilidade

Os três aspectos estão relacionados e devem ser considerados durante todo o desenvolvimento.

Por exemplo, implementar uma verificação de conflitos entre reservas contribui para a qualidade e a confiabilidade do sistema, reduzindo o risco de duplicidade.

Da mesma forma, organizar o código e utilizar tecnologias adequadas facilita a manutenção, contribuindo para a sustentabilidade tecnológica e econômica.

A tabela abaixo apresenta algumas dessas relações:

Medida	Qualidade	Redução de riscos	Sustentabilidade
Controle de acesso	Segurança	Acesso indevido	Proteção das informações
Verificação de conflitos	Confiabilidade	Reservas duplicadas	Melhor utilização dos recursos
Cópias de segurança	Integridade	Perda de informações	Continuidade do serviço
Interface acessível	Usabilidade	Erros de utilização	Inclusão dos usuários

Código organizado	Manutenibilidade	Dificuldade de manutenção	Evolução tecnológica
Digitalização das solicitações	Eficiência	Perda de registros físicos	Redução do consumo de papel

10.6. Considerações sobre qualidade, riscos e sustentabilidade

A análise de qualidade, riscos e sustentabilidade permitiu identificar fatores importantes para o desenvolvimento do Sistema de Reserva de Laboratórios e Equipamentos.

Os critérios de qualidade estabelecem condições para avaliar se o sistema atende às necessidades dos usuários. A identificação dos riscos permite antecipar possíveis problemas e definir medidas preventivas.

Já a sustentabilidade demonstra a importância de desenvolver uma solução que possa ser mantida e atualizada ao longo do tempo, considerando os custos, os impactos ambientais, os benefícios sociais e a evolução tecnológica.

Dessa forma, esses aspectos devem acompanhar todas as etapas do desenvolvimento, desde o planejamento até a futura manutenção do sistema.

10.7. Checklist da etapa

Qualidade, Riscos e Sustentabilidade

Concluído

- Definição dos critérios de qualidade.
- Identificação dos critérios de aceitação.
- Identificação dos principais riscos.
- Classificação de probabilidade e impacto.
- Definição de medidas preventivas.
- Análise da sustentabilidade econômica.
- Análise da sustentabilidade ambiental.
- Análise da sustentabilidade social.
- Análise da sustentabilidade tecnológica.
- Relação entre qualidade, riscos e sustentabilidade.

Próxima etapa: Plano Futuro de Desenvolvimento

A próxima etapa será a última parte do planejamento técnico do projeto.

Nela, vamos organizar as atividades necessárias para uma futura implementação, apresentando:

- Etapas de desenvolvimento e suas dependências.
- Distribuição de responsabilidades na equipe.
- Ferramentas e tecnologias previstas.
- Critérios de aceite para a entrega do sistema.
- Resultado esperado ao final do desenvolvimento.

Essa etapa encerrará o planejamento da solução antes da organização e revisão final do relatório técnico.

11. Plano Futuro de Desenvolvimento

Projeto: Sistema de Reserva de Laboratórios e Equipamentos

Nesta etapa, será apresentado um planejamento para uma futura implementação do Sistema de Reserva de Laboratórios e Equipamentos.

Até o momento, foram definidos o problema, os requisitos, os diagramas UML, os princípios de Orientação a Objetos, os padrões de projeto, a metodologia de desenvolvimento e os critérios de qualidade.

Com essas definições, é possível organizar as próximas atividades necessárias para transformar a proposta em um sistema funcional.

O planejamento considera o uso do Scrum, escolhido na etapa anterior, e divide o desenvolvimento em atividades menores, com responsabilidades e resultados esperados.

É importante destacar que este plano representa uma proposta de execução futura. O sistema ainda não foi implementado.

11.1. Etapas previstas para o desenvolvimento

O desenvolvimento será organizado em oito etapas principais.

Etapa	Atividade	Resultado esperado

1	Preparação do ambiente	Ambiente de desenvolvimento configurado.
2	Modelagem e banco de dados	Estrutura de armazenamento definida.
3	Autenticação e usuários	Cadastro, login e controle de acesso.
4	Gerenciamento de recursos	Cadastro de laboratórios e equipamentos.
5	Solicitação de reservas	Consulta de disponibilidade e criação de solicitações.
6	Análise das solicitações	Aprovação, rejeição e cancelamento.
7	Testes e correções	Funcionalidades verificadas e problemas corrigidos.
8	Implantação e manutenção	Sistema disponibilizado e preparado para evolução.

11.1.1. Descrição das etapas

Etapa 1 – Preparação do ambiente

Inicialmente, será necessário preparar o ambiente de desenvolvimento, instalar as ferramentas necessárias e organizar os arquivos do projeto.

Também serão definidos os padrões de organização do código e o repositório para controle de versões.

Etapa 2 – Modelagem e banco de dados

Será definida a estrutura de armazenamento das informações, considerando os usuários, recursos e reservas.

As tabelas deverão representar os relacionamentos definidos no Diagrama de Classes, mantendo a integridade dos dados.

Etapa 3 – Autenticação e usuários

Serão desenvolvidas as funcionalidades de cadastro, autenticação e controle de acesso.

Cada usuário terá um perfil, permitindo diferenciar as permissões de professores, funcionários e administradores.

Etapa 4 – Gerenciamento de recursos

Será desenvolvido o cadastro de laboratórios e equipamentos, permitindo consultar, atualizar, ativar e desativar os recursos.

Etapa 5 – Solicitação de reservas

Será implementada a funcionalidade principal do sistema, permitindo consultar a disponibilidade, selecionar um recurso e solicitar uma reserva.

Também será desenvolvida a verificação de conflitos de horário.

Etapa 6 – Análise das solicitações

Será implementado o ambiente administrativo para consultar solicitações pendentes, aprovar ou rejeitar pedidos e registrar o motivo da rejeição.

Também serão desenvolvidas as funcionalidades de acompanhamento e cancelamento.

Etapa 7 – Testes e correções

As funcionalidades serão testadas individualmente e em conjunto.

Serão verificados principalmente os controles de acesso, a integridade das informações, os conflitos de horário e o funcionamento das operações de reserva.

Etapa 8 – Implantação e manutenção

Após a aprovação dos testes, o sistema poderá ser disponibilizado para utilização.

Posteriormente, serão realizadas atividades de manutenção, correção de problemas e implementação de melhorias.

11.2. Dependências entre as etapas

As atividades possuem dependências porque algumas funcionalidades precisam estar concluídas antes que outras possam ser desenvolvidas.

Por exemplo, a solicitação de reservas depende da existência de usuários cadastrados e de recursos disponíveis.

A representação abaixo demonstra uma sequência simplificada dessas dependências:

#chatgpt-mermaid-_r_1pi_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(237, 237, 237);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_1pi_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1pi_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_1pi_ .error-icon{fill:rgb(48, 48, 48);}#chatgpt-mermaid-_r_1pi_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_1pi_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_1pi_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_1pi_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_1pi_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_1pi_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1pi_ .marker.cross{stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1pi_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_1pi_ p{margin:0;}#chatgpt-mermaid-_r_1pi_ .label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .cluster-label text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .cluster-label span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .cluster-label span p{background-color:transparent;}#chatgpt-mermaid-_r_1pi_ .label text,#chatgpt-mermaid-_r_1pi_ span{fill:rgb(237, 237, 237);color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .node rect,#chatgpt-mermaid-_r_1pi_ .node circle,#chatgpt-mermaid-_r_1pi_ .node ellipse,#chatgpt-mermaid-_r_1pi_ .node polygon,#chatgpt-mermaid-_r_1pi_ .node path{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .rough-node .label text,#chatgpt-mermaid-_r_1pi_ .node .label text,#chatgpt-mermaid-_r_1pi_ .image-shape .label,#chatgpt-mermaid-_r_1pi_ .icon-shape .label{text-anchor:middle;}#chatgpt-mermaid-_r_1pi_ .node .katex path{fill:#000;stroke:#000;stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .rough-node .label,#chatgpt-mermaid-_r_1pi_ .node .label,#chatgpt-mermaid-_r_1pi_ .image-shape .label,#chatgpt-mermaid-_r_1pi_ .icon-shape .label{text-align:center;}#chatgpt-mermaid-_r_1pi_ .node.clickable{cursor:pointer;}#chatgpt-mermaid-_r_1pi_ .root .anchor path{fill:rgb(175, 175, 175)!important;stroke-width:0;stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1pi_

.arrowheadPath{fill:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1pi_ .edgePath .path{stroke:rgb(175, 175, 175);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .flowchart-link{stroke:rgb(175, 175, 175);fill:none;}#chatgpt-mermaid-_r_1pi_ .edgeLabel{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_1pi_ .edgeLabel p{background-color:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1pi_ .edgeLabel rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1pi_ .labelBkg{background-color:rgba(0, 0, 0, 0.5);}#chatgpt-mermaid-_r_1pi_ .cluster rect{fill:rgb(48, 48, 48);stroke:rgba(255, 255, 255, 0.15);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .cluster text{fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ .cluster span{color:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ div.mermaidTooltip{position:absolute;text-align:center;max-width:200px;padding:2px;font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:12px;background:rgb(48, 48, 48);border:1px solid rgba(255, 255, 255, 0.15);border-radius:2px;pointer-events:none;z-index:100;}#chatgpt-mermaid-_r_1pi_ .flowchartTitleText{text-anchor:middle;font-size:18px;fill:rgb(237, 237, 237);}#chatgpt-mermaid-_r_1pi_ rect.text{fill:none;stroke-width:0;}#chatgpt-mermaid-_r_1pi_ .icon-shape,#chatgpt-mermaid-_r_1pi_ .image-shape{background-color:rgb(0, 0, 0);text-align:center;}#chatgpt-mermaid-_r_1pi_ .icon-shape p,#chatgpt-mermaid-_r_1pi_ .image-shape p{background-color:rgb(0, 0, 0);padding:2px;}#chatgpt-mermaid-_r_1pi_ .icon-shape .label rect,#chatgpt-mermaid-_r_1pi_ .image-shape .label rect{opacity:0.5;background-color:rgb(0, 0, 0);fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_1pi_ .label-icon{display:inline-block;height:1em;overflow:visible;vertical-align:-0.125em;}#chatgpt-mermaid-_r_1pi_ .node .label-icon path{fill:currentColor;stroke:revert;stroke-width:revert;}#chatgpt-mermaid-_r_1pi_ .node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_1pi_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_1pi_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_1pi_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_1pi_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_1pi_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1pi_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_1pi_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_1pi_ .node text{font-size:14px;font-weight:600;letter-

spacing:normal;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_1pi_ .edgeLabels text{font-size:13px;font-weight:600;letter-spacing:-0.08px;fill:rgb(153, 206, 255);}#chatgpt-mermaid-_r_1pi_ .node tspan[font-weight="normal"],#chatgpt-mermaid-_r_1pi_ .edgeLabels tspan[font-weight="normal"]{font-weight:600;}#chatgpt-mermaid-_r_1pi_ .edgeLabel .label rect{opacity:1;rx:13px;ry:13px;fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .node rect,#chatgpt-mermaid-_r_1pi_ .node circle,#chatgpt-mermaid-_r_1pi_ .node ellipse,#chatgpt-mermaid-_r_1pi_ .node polygon,#chatgpt-mermaid-_r_1pi_ .node path{fill:rgb(0, 40, 77);stroke:rgba(255, 255, 255, 0.1);stroke-width:1px;}#chatgpt-mermaid-_r_1pi_ .node rect{rx:16px;ry:16px;}#chatgpt-mermaid-_r_1pi_ .mermaid-decision .label-container{fill:rgb(0, 14, 26);stroke:rgb(26, 62, 95);stroke-dasharray:2,2;}#chatgpt-mermaid-_r_1pi_ .edgePaths .flowchart-link{stroke:rgb(175, 175, 175);stroke-width:1px;stroke-linecap:round;stroke-linejoin:round;}#chatgpt-mermaid-_r_1pi_ .marker{fill:rgb(175, 175, 175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_1pi_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}1. Preparação do ambiente2. Modelagem e banco dedados3. Autenticação e usuários4. Gerenciamento de recursos5. Solicitação de reservas6. Análise das solicitações7. Testes e correções8. Implantação e manutenção
Figura 5 – Dependências entre as etapas de desenvolvimento. Fonte: Elaborado pelo autor (2026).

Essa organização permite compreender quais atividades precisam ser concluídas antes do início de determinadas funcionalidades.

Embora algumas atividades possam acontecer em paralelo, as dependências principais devem ser respeitadas para evitar problemas durante a implementação.

11.3. Equipe e distribuição de responsabilidades

Para uma futura implementação, considera-se inicialmente uma equipe pequena, composta por quatro integrantes.

Cada integrante terá responsabilidades principais, mas poderá colaborar com os demais conforme as necessidades do projeto.

Integrante	Responsabilidade principal
Product Owner	Organizar as necessidades dos usuários e priorizar as funcionalidades.

Scrum Master	Acompanhar o processo de trabalho e auxiliar na remoção de impedimentos.
Desenvolvedor Front-end	Desenvolver as telas e os elementos de interação do sistema.
Desenvolvedor Back-end	Desenvolver as regras de negócio, autenticação e operações com os dados.

Além desses integrantes, os professores, funcionários e administradores da instituição poderão participar das avaliações do sistema, apresentando opiniões sobre as funcionalidades desenvolvidas.

Em uma equipe pequena, uma mesma pessoa poderá assumir mais de uma responsabilidade, desde que as atividades sejam organizadas adequadamente.

11.4. Ferramentas e tecnologias previstas

Para a implementação futura, são propostas algumas ferramentas e tecnologias que poderão auxiliar no desenvolvimento.

Ferramenta ou tecnologia	Finalidade
Java	Implementação das regras de negócio e conceitos de Orientação a Objetos.
Spring Boot	Desenvolvimento da aplicação e disponibilização das funcionalidades por meio de uma API.
PostgreSQL	Armazenamento e gerenciamento dos dados.
HTML, CSS e JavaScript	Desenvolvimento da interface do sistema.
Git	Controle de versões do código.

GitHub	Hospedagem do repositório e colaboração entre os desenvolvedores.
Visual Studio Code ou IntelliJ IDEA	Edição e desenvolvimento do código.
JUnit	Realização de testes automatizados.
Figma	Planejamento e prototipação das telas.

Essas tecnologias representam uma sugestão inicial. A escolha definitiva deverá considerar o conhecimento da equipe, os recursos disponíveis e as necessidades da instituição.

11.5. Critérios de aceite do sistema

Os critérios de aceite definem as condições que precisam ser atendidas para considerar as funcionalidades desenvolvidas como concluídas.

Para este projeto, foram definidos os seguintes critérios:

Funcionalidade	**Critério de aceite**
Login	O usuário consegue acessar o sistema com credenciais válidas.
Controle de acesso	Cada perfil consegue acessar somente as funcionalidades autorizadas.
Cadastro de recursos	O administrador consegue cadastrar e atualizar laboratórios e equipamentos.
Consulta de disponibilidade	O usuário consegue consultar os recursos e seus períodos disponíveis.
Solicitação de reserva	O sistema registra solicitações com informações completas e válidas.

Verificação de conflitos	O sistema impede solicitações conflitantes para o mesmo recurso e período.
Aprovação	O administrador consegue aprovar solicitações pendentes.
Rejeição	O administrador consegue rejeitar solicitações informando o motivo.
Histórico	O usuário consegue consultar suas próprias solicitações.
Cancelamento	O usuário consegue cancelar suas solicitações ou reservas conforme as regras definidas.
Segurança	As informações são protegidas por autenticação e permissões de acesso.
Desempenho	As operações principais atendem ao tempo de resposta estabelecido de até 3 segundos em condições normais.

Uma funcionalidade somente deverá ser considerada concluída após passar pelos testes correspondentes e atender aos critérios estabelecidos.

11.6. Cronograma estimado

Considerando o planejamento de cinco Sprints apresentado na etapa de Metodologia de Desenvolvimento, o projeto poderá ser executado inicialmente em dez semanas.

Período	Atividades principais
Semanas 1 e 2	Preparação, estrutura inicial e autenticação.
Semanas 3 e 4	Modelagem e gerenciamento de recursos.

Semanas 5 e 6	Solicitação de reservas e verificação de conflitos.
Semanas 7 e 8	Aprovação, rejeição, histórico e cancelamento.
Semanas 9 e 10	Testes, correções e preparação para implantação.

Tabela 8 – Cronograma estimado de desenvolvimento. Fonte: Elaborado pelo autor (2026).

O cronograma poderá sofrer alterações conforme a complexidade das funcionalidades, a disponibilidade dos integrantes e os resultados obtidos durante as Sprints.

11.7. Resultados esperados

Ao final do desenvolvimento, espera-se obter um sistema que permita centralizar e organizar as reservas de laboratórios e equipamentos da instituição.

Entre os principais resultados esperados estão:

- Redução de conflitos entre solicitações de reservas.
- Maior organização das informações sobre laboratórios e equipamentos.
- Facilidade para consultar disponibilidade.
- Controle das solicitações por meio de aprovação e rejeição.
- Acompanhamento das reservas pelos usuários.
- Maior segurança no acesso às informações.
- Redução da dependência de planilhas e controles manuais.
- Facilidade para realizar futuras melhorias no sistema.

Esses resultados são objetivos esperados da proposta e deverão ser avaliados após uma futura implementação.

11.8. Possibilidades de evolução futura

Depois que as funcionalidades principais estiverem implementadas e funcionando corretamente, o sistema poderá receber novas melhorias.

Algumas possibilidades são:

1. Notificações automáticas: envio de avisos por e-mail quando uma solicitação for aprovada, rejeitada ou cancelada.

2. Relatórios administrativos: geração de relatórios sobre a utilização dos laboratórios e equipamentos.

3. Integração com calendário: possibilidade de visualizar as reservas em uma agenda.

4. Aplicação móvel: desenvolvimento de uma versão adaptada para dispositivos móveis.

5. Controle de utilização: registro de informações sobre a utilização efetiva dos recursos reservados.

Essas funcionalidades não fazem parte do escopo inicial, mas poderão ser consideradas em versões futuras, conforme as necessidades identificadas.

11.9. Considerações sobre o plano futuro

O planejamento apresentado organiza as principais atividades necessárias para uma futura implementação do Sistema de Reserva de Laboratórios e Equipamentos.

A divisão do desenvolvimento em etapas permite estabelecer uma sequência de trabalho, identificar dependências e distribuir responsabilidades entre os integrantes da equipe.

A definição das ferramentas, do cronograma e dos critérios de aceite também contribui para tornar o processo mais organizado e facilitar o acompanhamento dos resultados.

Com esse planejamento, a proposta apresenta uma direção para transformar os requisitos e modelos desenvolvidos anteriormente em uma solução funcional.

11.10. Checklist da etapa

Plano Futuro de Desenvolvimento

Concluído

- Definição das etapas de desenvolvimento.
- Identificação das dependências.
- Distribuição das responsabilidades.
- Seleção de ferramentas e tecnologias.
- Definição dos critérios de aceite.

- Elaboração do cronograma.
- Identificação dos resultados esperados.
- Proposição de melhorias futuras.

Encerramento do desenvolvimento da proposta

Com esta etapa, concluímos as partes principais do planejamento técnico do projeto Sistema de Reserva de Laboratórios e Equipamentos.

O trabalho foi desenvolvido seguindo uma sequência que parte da identificação de um problema e chega ao planejamento de uma possível implementação.

As etapas elaboradas foram:

Etapa	Conteúdo	Situação
1	Contexto e definição do problema	Concluída
2	Levantamento e especificação de requisitos	Concluída
3	Diagrama de Casos de Uso	Concluída
4	Diagrama de Classes	Concluída
5	Diagrama de Sequência	Concluída
6	Diagrama de Atividades	Concluída
7	Orientação a Objetos	Concluída
8	Padrões de Projeto	Concluída
9	Metodologia de Desenvolvimento	Concluída

10	Qualidade, Riscos e Sustentabilidade	Concluída
11	Plano Futuro de Desenvolvimento	Concluída

Próxima atividade: organizar e revisar o relatório técnico final, verificar a coerência entre os diagramas e os requisitos, acrescentar referências bibliográficas, declaração de autoria e demais informações solicitadas pela faculdade, e preparar o documento em PDF no formato exigido.

Essa revisão é importante porque, embora as etapas estejam desenvolvidas, alguns detalhes precisam permanecer consistentes entre si — especialmente os requisitos, as classes, os padrões de projeto e o cronograma.